



Mirko Mandic, Inc.
Phone: 863 7828
Fax: 863 7828
Internet: www.mirkomandic.com
Intranet: www.mirkomandic.com/iss

REQUEST FOR INFORMATION SYSTEM SERVICES

DATE OF REQUEST	SERVICE REQUESTED FOR DEPARTMENT
03/14/2002	Member Services

SUBMITTED BY (key user contact)	EXECUTIVE SPONSOR (funding authority)
Name Nikola Pejovic Title System Analyst, Mirko Mandic, Inc. Office WROB Phone 863 7828	Name Mirko Mandic Title Owner, Mirko Mandic, Inc. Office WROB Phone 863 7828

TYPE OF SERVICE REQUESTED:

- | | |
|---|---|
| ☒ Information Strategy Planning | ☐ Existing Application Enhancement |
| ☐ Business Process Analysis and Redesign | ☐ Existing Application Maintenance (problem fix) |
| ☒ New Application Development | ☐ Not Sure |
| ☐ Other (please specify _____) | |

BRIEF STATEMENT OF PROBLEM, OPPORTUNITY, OR DIRECTIVE

Mr. Mirko Mandic is a passionate CD collector. His rapidly growing collection consists of about 300 titles. As the number of CDs is increasing, it is starting to be hard for Mr. Mandic to keep track of relevant album – related information (album's title, artist / group's name, original release date, label, and manufacturer catalog number). He needs a database, which can store the information and allow him to manipulate with it effectively. Mr. Mandic also wants to be able to access the database indirectly, through a GUI. He does not want the interface to be overly complex, but rather, simple and easy to interact with. The GUI should allow user to add a new record to the database, search the database, and update or delete an already existing record.

BRIEF STATEMENT OF EXPECTED SOLUTION

An application, which will have the following characteristics will be developed:

- user-friendly front end, a "bridge" between a user and a database,
- connectivity between the GUI and the database,
- efficient utilization of the query capabilities of the database, which would allow the user to manipulate with the data easily.

ACTION (ISS Office Use Only)

☐ Feasibility assessment approved

Assigned to _____

☐ Feasibility assessment waived

Approved Budget \$ _____

Start Date _____ Deadline _____

☐ Request delayed

Backlogged until date: _____

☐ Request rejected

Reason: _____

Authorized Signatures: _____

Project Executive Sponsor

PROBLEM STATEMENT MATRIX

PROJECT: CD Collection System	PROJECT MANAGER: Mirko Mandic
CREATED BY: Mirko Mandic, Inc.	LAST UPDATED BY: Mirko Mandic
DATE CREATED: 03/14/2002	DATE LAST UPDATED: 03/14/2002

Brief Statements of Problem, Opportunity, or Directive	Urgency	Visibility	Annual Benefits	Priority or Rank	Proposed Solution
1. Currently, the only way for a CD collector to find a title (and other corresponding information about an album) within the collection is to physically search the collection, which can be very time-consuming, frustrating and impossible if the collector is away from the collection.	2 months	High	Unknown	1	New Development
2. Currently, there is no efficient CD collection content – reporting capabilities.	2months	Medium	Unknown	2	New Development
3. Currently, it is very hard to sort CDs because, depending on circumstance, different sorting criteria (alphabetically by album name, alphabetically by artist/group name, chronologically by release date) can be used.	2 months	Medium	Unknown	2	New Development
4. There is an opportunity for placing the database on a web server, so it can be accessed from anywhere on the web.	6 months	Low - out of immediate scope of this project	Unknown	3	New Development

CD COLLECTION PROJECT FEASIBILITY ASSESSMENT REPORT

I. Executive Summary

A. Summary of Recommendation

New development of an entirely computer-based system is recommended. There are numerous problems (please see Problem Statement Matrix) associated with the way that Mr. Mandic currently handles his growing CD collection. The new CD collection system would allow the user to store all the relevant album-related data (manufacturer catalog number, album name, artist/group name, label, release date), and manipulate with the data easily. Manipulation with the data would include – adding a new album, finding an already stored album, updating album info, and removing an album from the collection. According to Mr. Mandic, the user interface should allow him to do only these actions, and do them quickly and easily.

B. Anticipated Benefits

There are numerous anticipated benefits associated with the development of the new system. Mr. Mandic would be able to access the album information even if the CD cannot be accessed physically. Also, much time would be conserved during this process because he wouldn't have to physically search through the hundreds of CDs but rather, just type in his search criteria. Also, the user would be able to sort the CDs according to numerous criteria (release date, alphabetically by artist name, etc.) with just one mouse-click. The collector would also be reminded by browsing through the database that a borrowed CD is missing after the specified return time.

C. Report Contents

No paper-based reports will be generated. However, system will feature on-screen reports on information about each of the CDs within the collection. The information reported to the collector will be:

Manufacturer catalog number

Album name

Artist/group name

Label

Original release date

II. Background Information

A. Description of Project Request

A request has been put forth by Mr. Mirko Mandic, who is a passionate CD collector. Mr. Mandic's CD collection is rapidly growing, and currently, it includes around 300 titles. In his request, Mr. Mandic has stated that the lack of adequate database for his CDs makes his hobby much less enjoyable than it could be. According to Mr. Mandic, it has been increasingly hard to deal with the CDs and the relevant information. He wants the system to feature a graphical, user-friendly front end that would allow him to browse through his collection and update relevant information accordingly.

B. Survey Phase Activities

1. Project Context

Mr. Mandic contacted one of our system analysts, Nikola Pejovic. Mr. Mandic specified his problem, and requested the investigation of possible solutions. He also stated very clearly what the new system's major characteristics should be.

2. Project Scope

The customer desires a system that would have a Windows style user interface. The interface should reflect all of the database functionalities (add, find, update, remove), but should not have any other additional features that would make the use harder. The customer has a limited budget for this project of \$5,000.00. Project team investigated the possibility of purchasing an off-the-shelf package, but none were found within the customer's budget.

3. Project Team

Project Manager: Mirko Mandic

Systems Analyst: Nikola Pejovic (user contact)

4. Project Schedule

The project must be completed by April 20, 2002.

III. Detailed Recommendation

A. Narrative Recommendation

1. Immediate Fixes

A personal computer with standard office software and printer will provide a means to record and store data, and provide the reporting capabilities for the system.

2. Quick Fixes

In the desired allotment of time, the customer will be equipped with the system as described in the previous sections.

3. Enhancements

Once the proposed system has been developed, additional enhancements can be added including Internet access (placing the system on a web server), allowing for more than one user, etc.

4. New Systems Development

The entire system will be new development.

B. Project Plan

1. Initial Project Objectives

- a. The new system must be entirely computer-based.
- b. The new system must provide a GUI.
- c. The database storing the information will be relational.
- d. Reduce operational time by at least 90% in searching through the albums and their sorting.

2. Project Development Timeline

- System Analysis: March 1 – March 15
- System Design: March 15 – April 5
- System Implementation: April 5 – April 15
- System Support and Enhancements: April 5 –

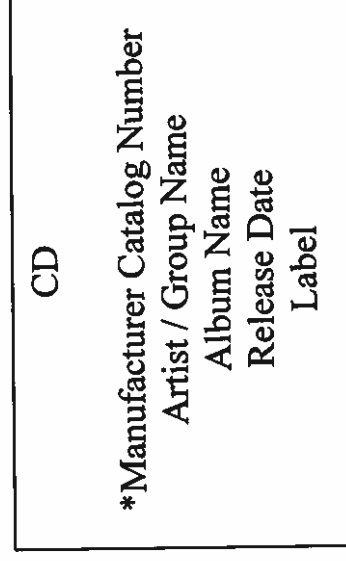
PROBLEMS, OPPORTUNITIES, OBJECTIVES AND CONSTRAINTS MATRIX

Project: CD Collection System	Project Manager: Mirko Mandic
Created by: Mirko Mandic	Last Updated by: Mirko Mandic
Date Created: 03/15/2002	Date Last Updated: 03/15/2002

CAUSE AND EFFECT ANALYSIS		SYSTEM IMPROVEMENT OBJECTIVES	
Problem or Opportunity	Causes and Effects	System Objective	System Constraint
1. There is no database for the relatively big, quickly growing CD collection. 2. There is an opportunity for placing the newly built database on a web server. However, this opportunity is out of the immediate scope of this project.	1. A collector can access album information only if the CD can be accessed physically. 2. It is almost impossible to sort the CDs, because depending on circumstances, different sorting criteria might be needed. 3. There is no way for the collector to view all the titles (and other related information) next to each other, in a list. 4. It is very hard to timely notice that there is a missing album if one is lost or stolen. 5. If placed on a web server, the database could be accessed from anywhere in the world.	1. Provide an interface for user's interaction with a database. 2. Electronically record all the CDs and store them in a relational database. 3. Allow user to add an album, find an already recorded album, update information about an already recorded album, or delete an album from the database. 4. Reduce operational time by at least 90% in searching through the albums and their sorting.	1. The new system must be built by April 20, 2002. 2. The new system can not cost more than \$5000. 3. The interface built during the construction phase of the development cycle must be user-friendly, graphical one.

CD COLLECTION ER DIAGRAM

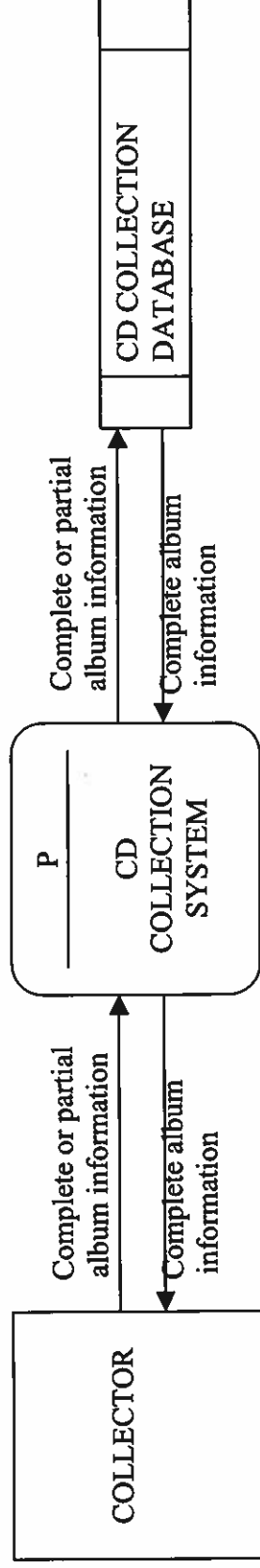
There is only one entity, CD, pertinent to the CD Collection system. Since this project is built specifically for the needs of a single user (Mr. Mandic), the collector information need not be stored. However, if the scope of the project was expanded to include multiple collectors, it would include another entity (Collector) describing each of the users (Appendix A).



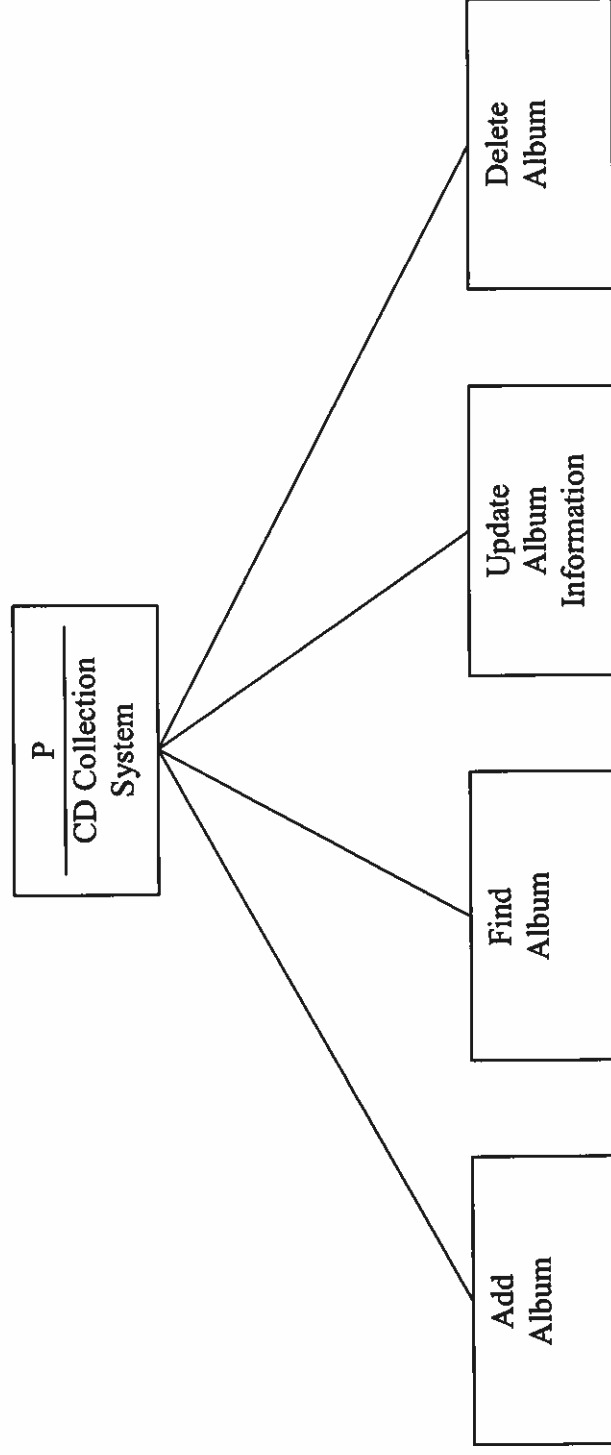
Entity/Definition Matrix

ENTITY	BUSINESS DEFINITION
COLLECTOR	A person who gathers musical albums for his/hers own pleasure and/or profit. Since the system is designed for a specific user (Mr. Mandic), Collector entity will represent this user throughout the documentation.
CD	Compact Disk. A musical album, stored in digital format.

CONTEXT DIAGRAM



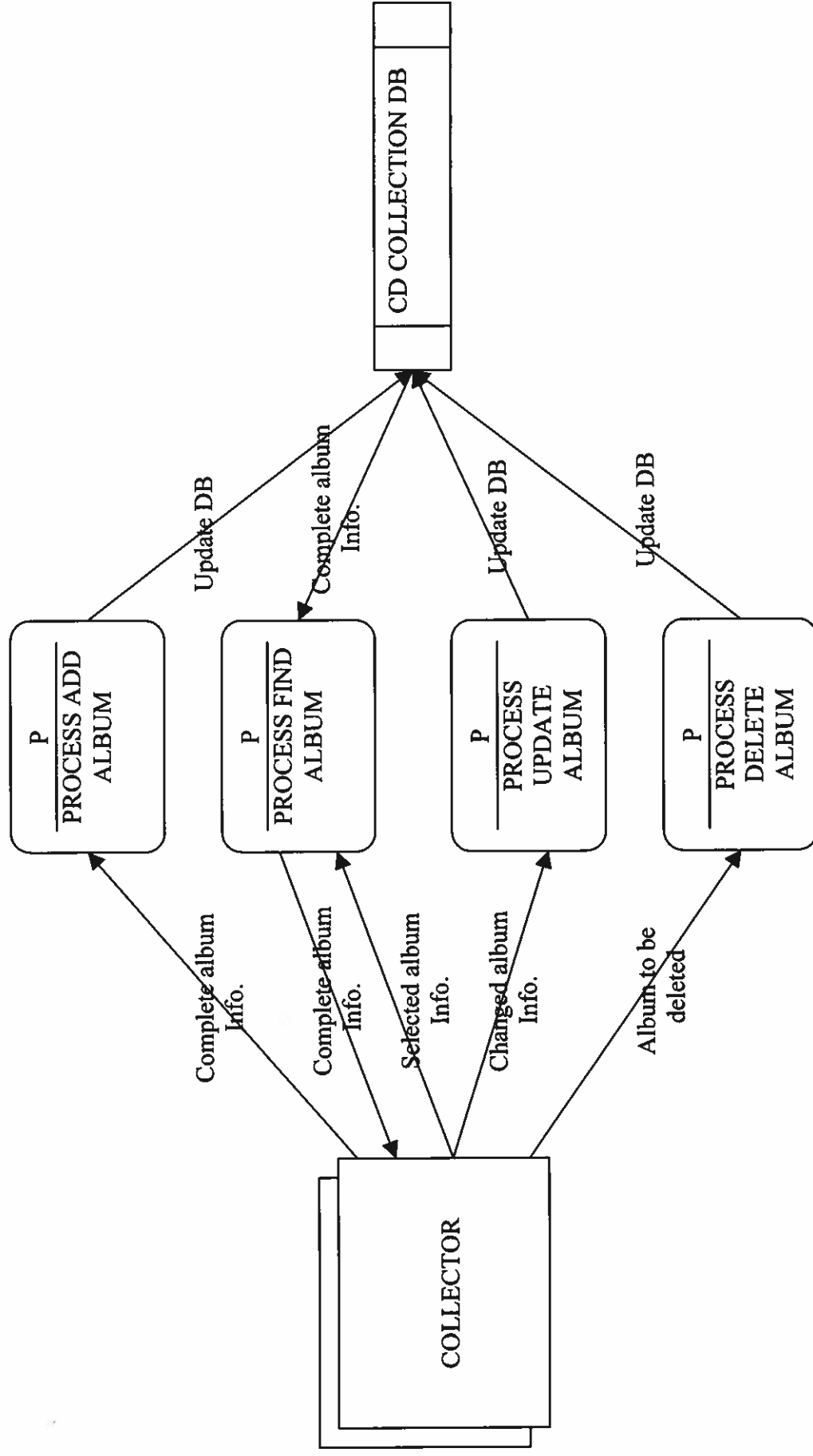
EVENT DECOMPOSITION DIAGRAM



EVENT RESPONSE LIST

Event Description	Trigger (Inputs)	Responses (Outputs)
Collector adds complete album information.	Complete album data (album name, artist/group name, catalog number, release date, label)	Update CD Collection DB
Collector searches for a particular album.	Selected album data – search criteria	Complete album info displayed if album is found “No record found” message displayed if album isn’t found
Collector updates album information.	Album data selected by collector	Update CD Collection DB
Collector deletes album from the collection.	Complete album data	Update CD Collection DB

CD COLLECTION SYSTEM DIAGRAM



UPDATING ALBUM INFORMATION POLICY

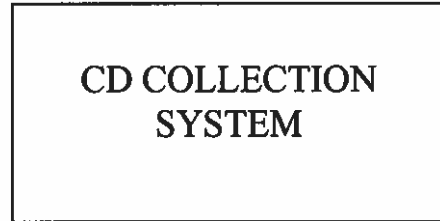
CONDITIONS:

Connection established	Y	Y	Y	N
Album found	Y	Y	N	-
Album info. changed	Y	N	-	-

ACTIONS:

Update album info.	X			
Don't update album info.		X	X	X

**CD COLLECTION SYSTEM
LOCATION CONNECTIVITY DIAGRAM**



SYSTEM PROPOSAL FOR CD COLLECTION

I. Introduction

A. Purpose of Report

This proposal will offer two candidate system solutions. It will also elaborate on each proposal candidate's characteristics and recommend the solution which best meets the customer's requirements.

B. Background of the Project

One assumption must be taken into consideration when analyzing this proposal: the customer has pre-purchased some system hardware and this proposal does not include those costs, other than those of certain input and output devices (please see the attached System Candidate Matrix). The system must be entirely computerized, and it must feature a GUI. After an extensive system analysis phase identified problems, opportunities, objectives and constraints of the new system, several suggestions for data, process, and network modeling were developed.

C. Scope of the Project

The project entails recognition of two candidate solutions and their characteristics (hardware and software needed, and benefits achieved by those particular choices), as well as comparison of these solutions with regard to their operational, technical, economic and schedule feasibility.

D. Structure of the Report

This report includes:

- description of the generated solution,
- feasibility analysis, based on cost-benefit relationship,
- description of the system's requirements,
- comparison of alternative solutions,
- recommendations,
- System Candidate Matrix and
- Feasibility Matrix

II. Tools and Techniques Used

A. Solution Generated

The recommended solution is identified and generated through the detailed examination of the following characteristics of the two candidate systems:

- portion of the system computerized,
- business benefits that would be realized,
- servers and workstations needed,
- software tools needed to design and build the candidate,
- application software needed,
- method of data processing used,
- output devices that would be used, special output requirements, and output considerations, and
- input devices that would be used, special input requirements, and input considerations

B. Feasibility Analysis (Cost-Benefit)

Feasibility analysis is based on the following four categories for each candidate:

- operational feasibility (functional and political feasibility – a description of how well received this solution would be from user management, user and organization perspective) – 30%
- technical feasibility (technology and expertise) – 30%
- economic feasibility – 30%
- schedule feasibility – 10%

III. Information System Requirements

The entire CD Collection system must be computerized. The customer has requested that the system utilize GUI front end. The system must be supported by single server/client PC with the Windows – generation operating system and must have a relational database. Software tools could include any of the standard languages (JAVA, C++, Visual Basic, etc.). The system must provide data back-up hardware and software. The method of data processing will be online, real-time. Output devices required by the system are a standard printer and an external zip drive for archiving and data backup purposes. There will be no special input requirements.

IV. Alternative Solutions and Feasibility Analysis

Two alternative solutions have been selected for consideration. Their specific characteristics are listed in the attached System Candidate Matrix and Feasibility Analysis.

V. Recommendations

Clearly, Candidate 1 is a superior solution. Actually, it appears that this candidate is the perfect solution for the target system, since its overall score is 100% when all of the criteria are considered. Candidate 2 has an overall score of 77%, caused by some of the following: too complex for specific customer needs, poor economic feasibility (it costs 2 times more than what the customer's budget allows) and extremely low schedule feasibility (it would take 6 to 9 months to implement it, comparing to 2, what is required for Candidate 1).

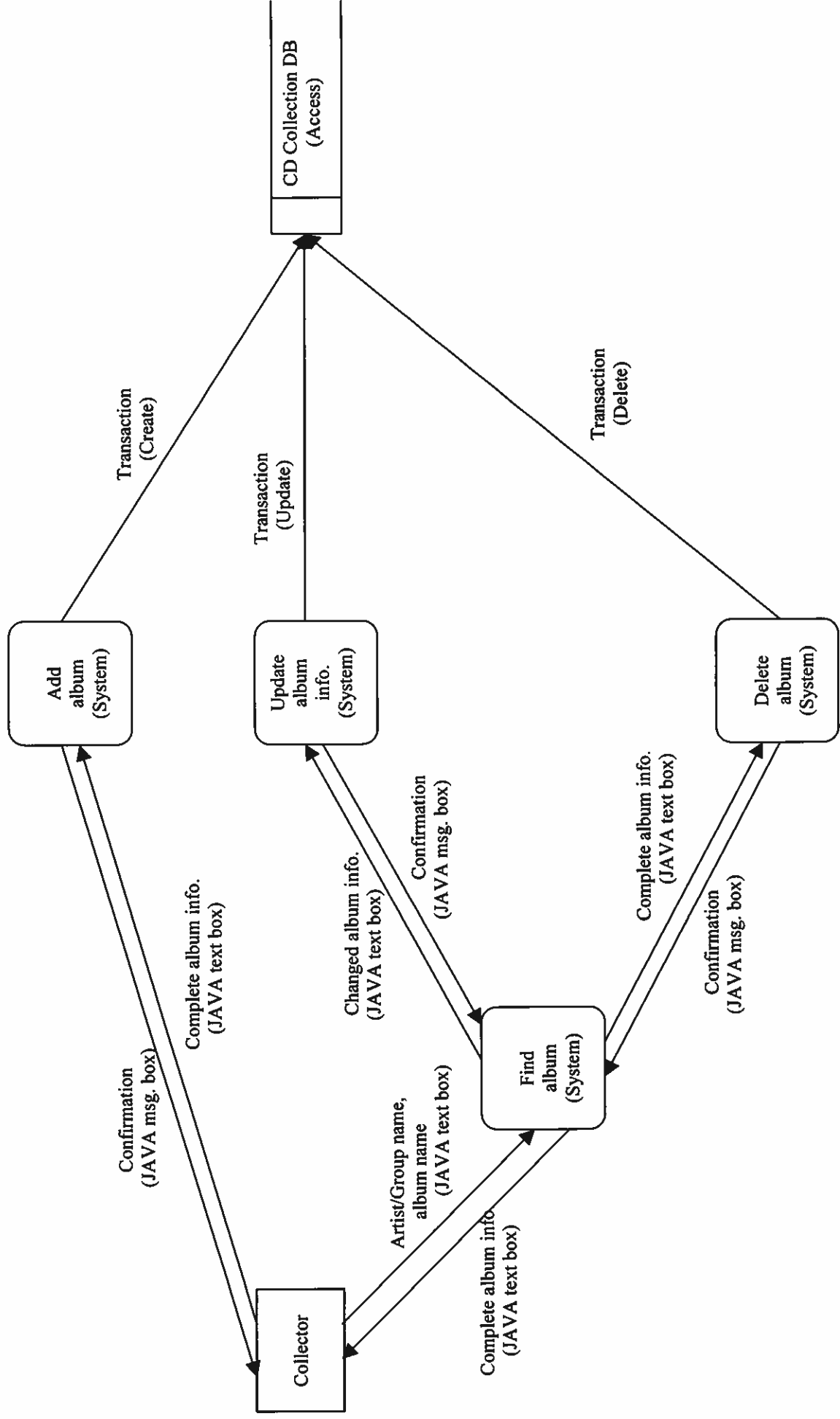
CD COLLECTION SYSTEM CANDIDATE MATRIX

Characteristics	Candidate 1	Candidate 2
Portion of System Computerized Brief description of that portion of the system that would be computerized in this candidate.	The entire CD Collection system will be computerized.	Same as Candidate 1.
Benefits Brief description of the business benefits that would be realized for this candidate.	All customer's requirements will be met. No additional modules are included.	All customer's functional requirements will be met. However, the interface will be overly complex and its functionalities will exceed customer's requirements
Servers and Workstations A description of the servers and workstations needed to support this candidate.	Single server with MS Windows 2000.	Same as Candidate 1.
Software Tools Needed Software tools needed to design and build the candidate (e. g., database management system, emulators, operating systems, languages, etc.). Not generally applicable if applications software packages are to be purchased.	JAVA and MS Access. These tools will provide the greatest development flexibility for the Candidate. JDBC – ODBC bridge will utilize the capabilities of the database most efficiently. JAVA will also be used for creation of the GUI that will completely meet customer's requirements.	Application software package is to be purchased.
Application Software A description of the software to be purchased, built, accessed, or some combination of these techniques.	CD Collection system will be built in-house.	Third party vendor packaged solution.
Method of Data Processing Generally some combination of: on-line, batch, deferred batch, remote batch, and real-time.	Real-time data processing.	Same as Candidate 1.
Output Devices and Implications A description of output devices that would be used, special output requirements, (e.g. network, preprinted forms, etc.), and output considerations (e.g., timing constraints).	15" LCD Display. HP All-In-One printer. Standard 8x10 paper, color cartridges.	15" LCD Display. HP Laser Jet printer. Standard 8x10 paper, color cartridges.
Input Devices and Implications A description of Input methods to be used, input devices (e.g., keyboard, mouse, etc.), special input requirements, (e.g. new or revised forms from which data would be input), and input considerations (e.g., timing of actual inputs).	Keyboard. Mouse. No special input requirements. Inputs will be made as additions are made to the collection.	Keyboard. Mouse. Bar-Code reader for scanning CD's UPC. Inputs will be made as additions are made to the collection.
Storage Devices and Implications Brief description of what data would be stored, what data would be accessed from existing stores, what storage media would be used, how much storage capacity would be needed, and how data would be organized.	Following data will be stored: CD catalog number, album name, artist/group name, label, release date, and will all be stored in single MS Access DB table. Primary storage device will be standard (20 GB) hard drive. External zip drive will be needed for data backup purposes.	Same as Candidate 1.

CD COLLECTION FEASIBILITY MATRIX

Feasibility Criteria	Wt.	Candidate 1	Candidate 2
Operational Feasibility Functionality. A description of to what degree the candidate would benefit the organization and how well the system would work. Political. A description of how well received this solution would be from both user management, user, and organization perspective.	30%	Supports all customer requested functionality. Also, manages to do so through a front end that completely satisfies all customer criteria. Since the system is built through a close cooperation with all the important players from the user side, it will be received very well by user management, user and organization.	Expands upon original requirements, being overly complex for specific customer needs.
		Score: 100	Score: 90
Technical Feasibility Technology. An assessment of the maturity, availability (or ability to acquire), and desirability of the computer technology needed to support this candidate. Expertise. An assessment to the technical expertise needed to develop, operate, and maintain the candidate system.	30%	System will be written in standard language (JAVA), one that is widely regarded as a language that is here to stay. Future upgrade compatibility is high. In-house application development for entire project.	System utilizes standard Windows API. This will insure future operating system upgrade compatibility. Makers of the system offer complete support in the areas of system operating and maintenance. However, possible unexpected hazards will mean loss of time.
		Score: 100	Score: 90
Economic Feasibility Cost to develop: Payback period (discounted): Net present value: Detailed calculations:	30%	\$5,000.00 1 year \$4,450.00 See Appendix B.	\$13,000.00 3 years \$9,767.00 See Appendix B.
		Score: 100	Score: 60
Schedule Feasibility An assessment of how long the solution will take to design and implement.	10%	Less than 2 months	6 to 9 months
		Score: 100	Score: 50
Ranking:	100%	100	77

CD COLLECTION PHYSICAL DATA FLOW DIAGRAM



ENTITIES TO THIRD NORMAL FORM

As it was previously indicated in the CD Collection ER Diagram, there is only one entity, CD, needed for the implementation of the CD Collection system, as specified by the customer, Mr. Mandic. However, possible expansion of the system to include more than one collector would lead to different entity relations and their normalization (Appendix C).

CD (*Manufacturer Catalog Number, Artist/Group Name, Album Name, Release Date, Label)

COMPLETE TABLE DEFINITIONS

CD Collection database includes only one table - CD. Potential expansion of the project scope would lead to inclusion of one more table - Collector (Appendx D). CD table consists of the following fields:

Entity Name: Manufacturer Catalog Number (Primary Key)
Data Type: String
Data Mask: xxxxxxxxxxxxxxxx
Value Range: 1 to 15 characters
Default Value: blank
Entity Description: Manufacturer Catalog Number is the unique number specified by the label at the time of release.

Entity Name: Artist/Group Name(Secondary Key)
Data Type: String
Data Mask: xx
Value Range: 1 to 30 characters
Default Value: blank
Entity Description: Name of the group/artist who made the album.

Entity Name: Album Name (Secondary Key)
Data Type: String
Data Mask: xx
Value Range: 1 to 30 characters
Default Value: blank
Entity Description: Title of the album.

Entity Name: Release Date (Secondary Key)
Data Type: String
Data Mask: xxxxxxxxxxxxxxxxxxxxxxxx
Value Range: 2 to 20 characters
Default Value: blank
Entity Description: The original album release date. Some CDs don't provide this information (especially re-issues) and in that case, a descriptive release date can be used, i.e. "late sixties".

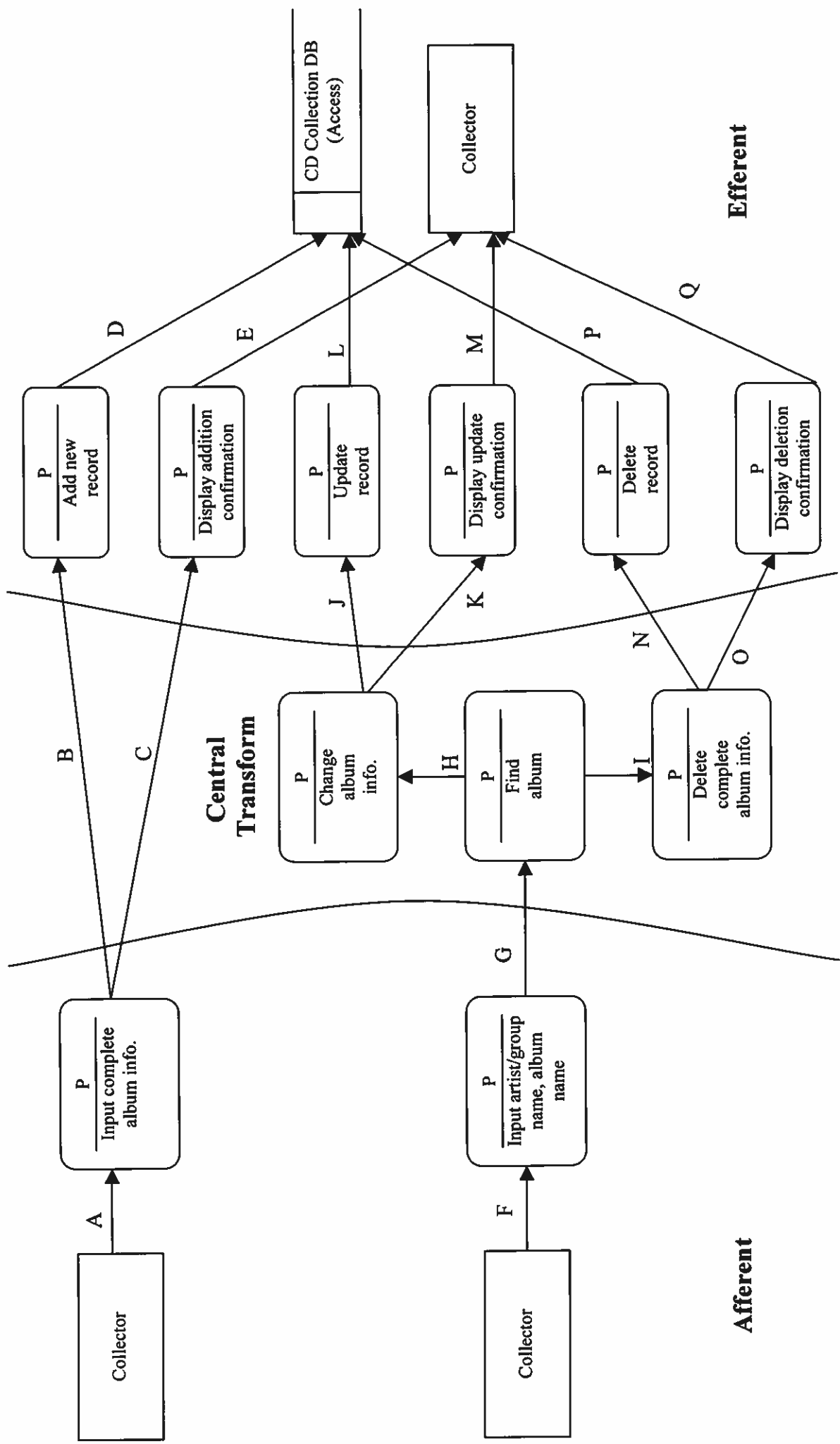
Entity Name: Label (Secondary Key)
Data Type: String
Data Mask: xxxxxxxxxxxxxxxxxxxxxxxx

Value Range: 1 to 20 characters

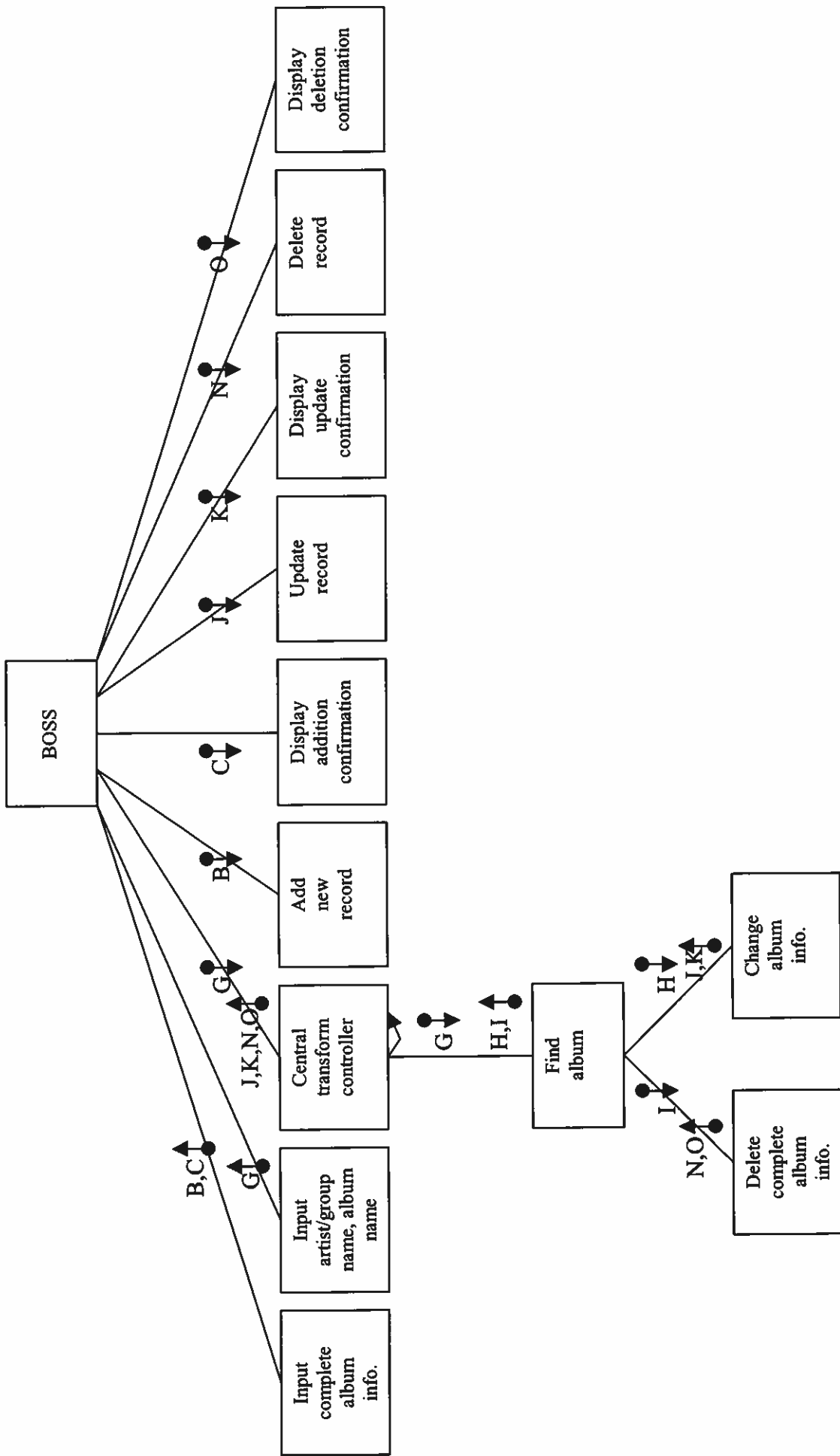
Default Value: blank

Entity Description: Label under which the album was released.

CD COLLECTION PARTITIONED DFD

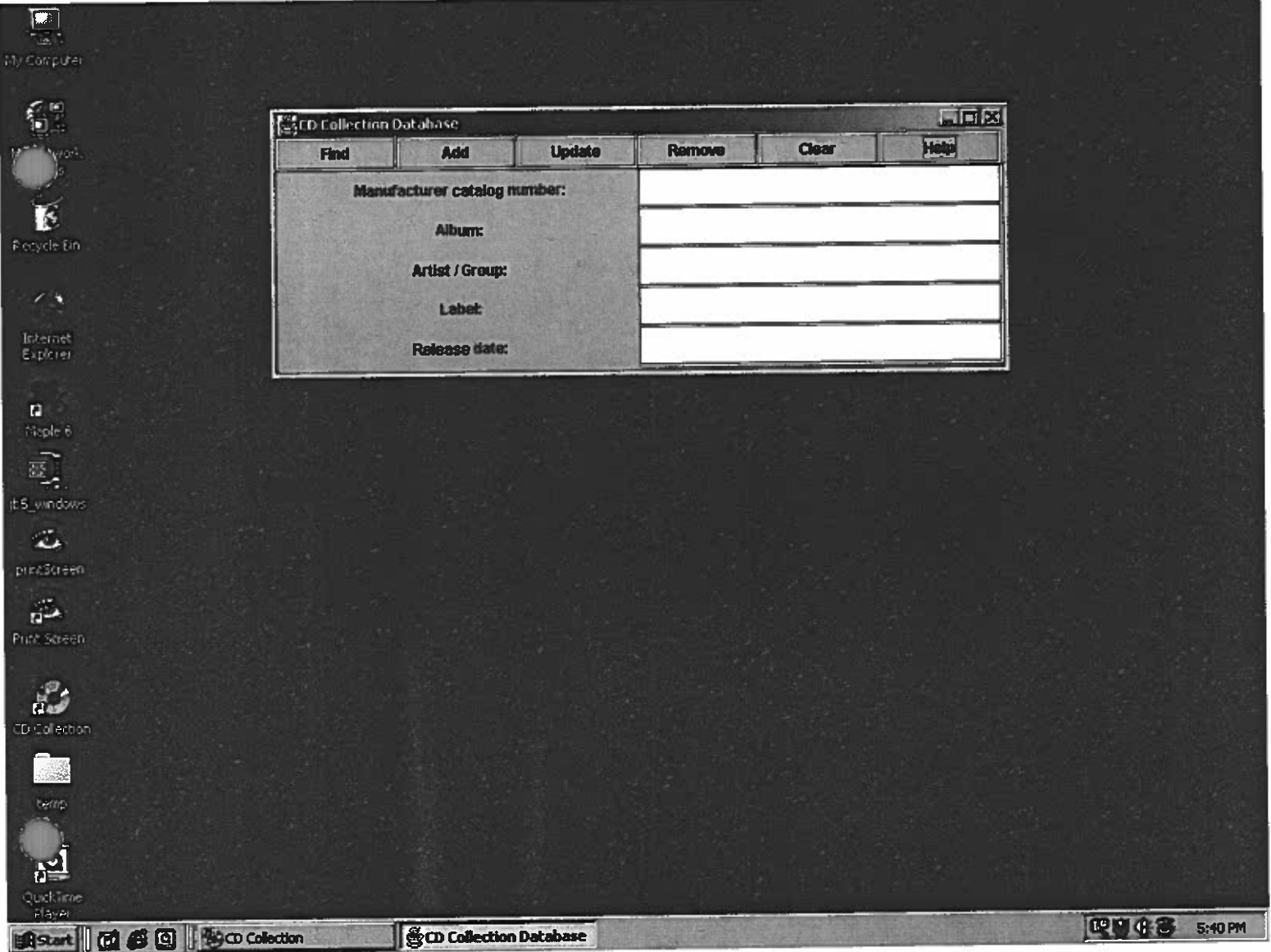


CD COLLECTION STRUCTURE CHART



CD COLLECTION INPUT/OUTPUT SCREENS

The following documentation illustrates the “look and feel” of the CD Collection system. All of the system’s functionalities are described through a detailed step-by-step example. Therefore, it can also be viewed as the user’s manual for the system. Please note that the system cannot be run if the database is not registered as an ODBC source (Appendix E).



- My Computer
- My Network Places
- Recycle Bin
- Internet Explorer
- Maple 6
- it5_windows
- printScreen
- Print Screen
- CD Collection
- temp
- QuickTime Player

CD Collection Database

FindAddUpdateRemoveClearHelp

Manufacturer catalog number:

Album:

Artist / Group:

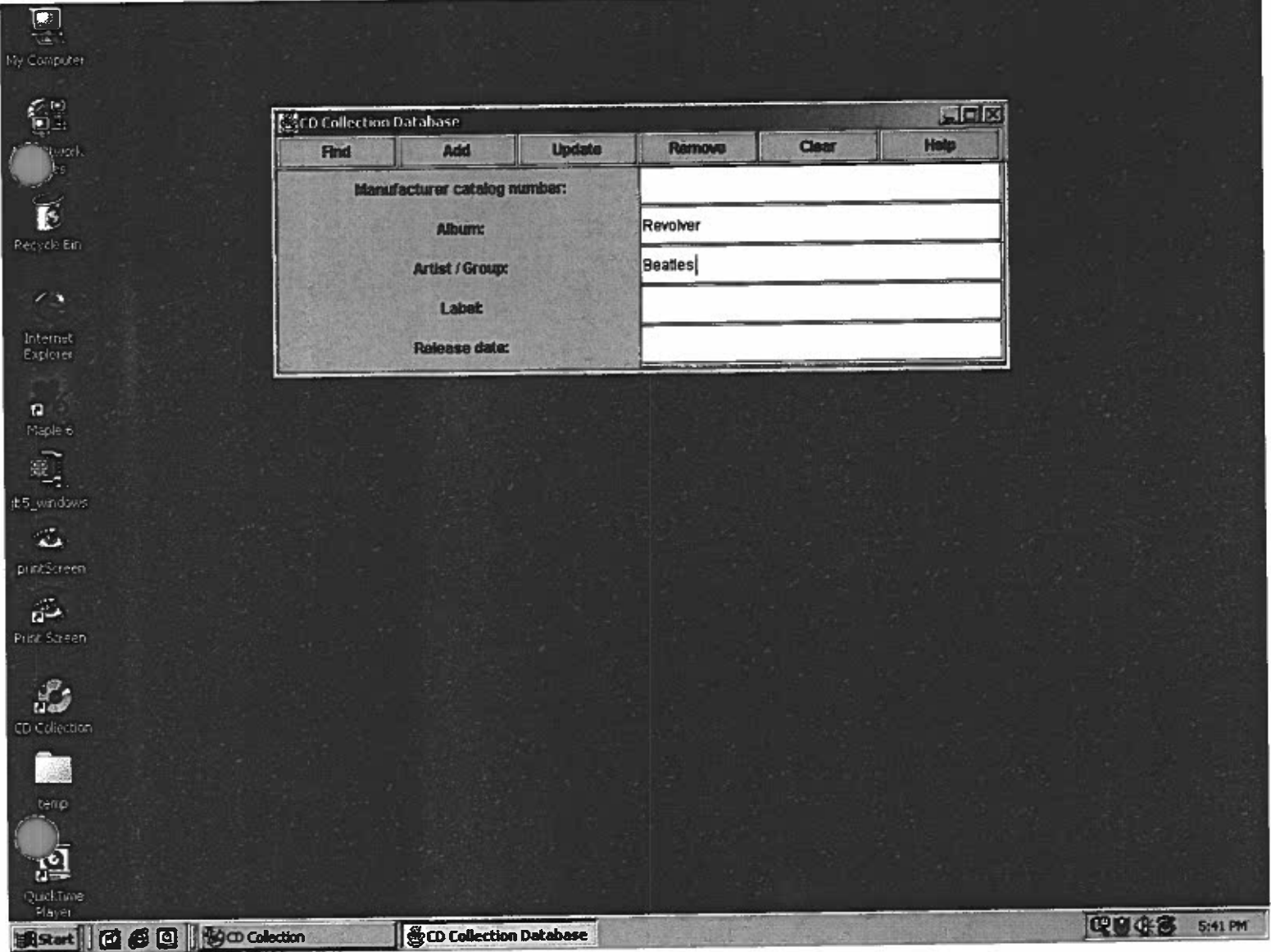
Label:

Release date:

CD Collection Database

Click Find to locate a record.
Click Add to insert a new record.
Click Update to update the information in a record.
Click Delete to delete a record.
Click Clear to empty the textfields.

OK



- My Computer
- Recycle Bin
- Internet Explorer
- Maple 6
- 5_windows
- printScreen
- Print Screen
- CD Collection
- temp
- QuickTime Player

CD Collection Database

Find	Add	Update	Remove	Clear	Help
Manufacturer catalog number:			You must specify catalog number		
Album:			Revolver		
Artist / Group:			Beatles		
Label:					
Release date:					

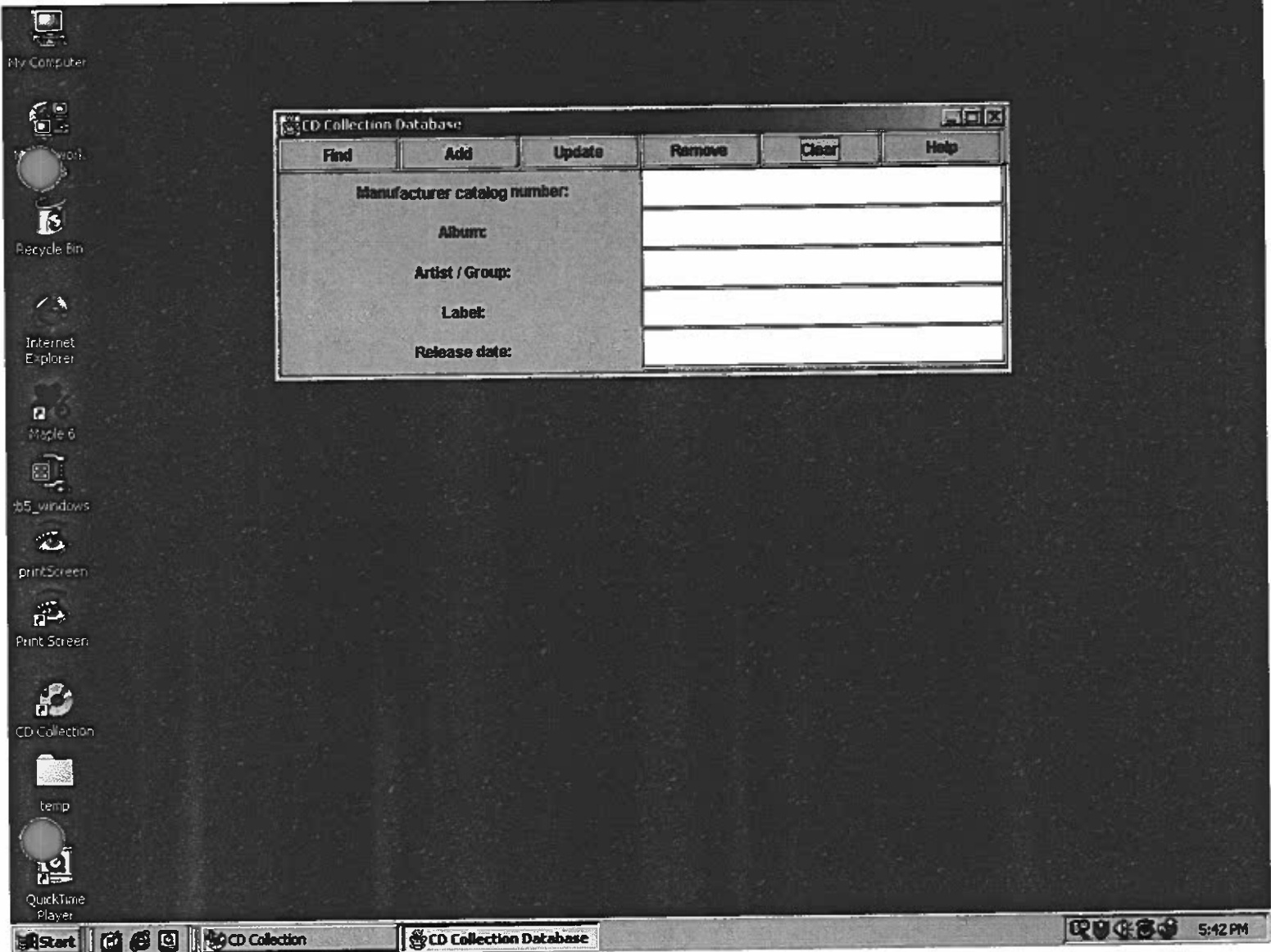
- My Computer
- work
- Recycle Bin
- Internet Explorer
- Maple 6
- ts_windows
- printScreen
- Print Screen
- CD Collection
- temp
- QuickTime Player

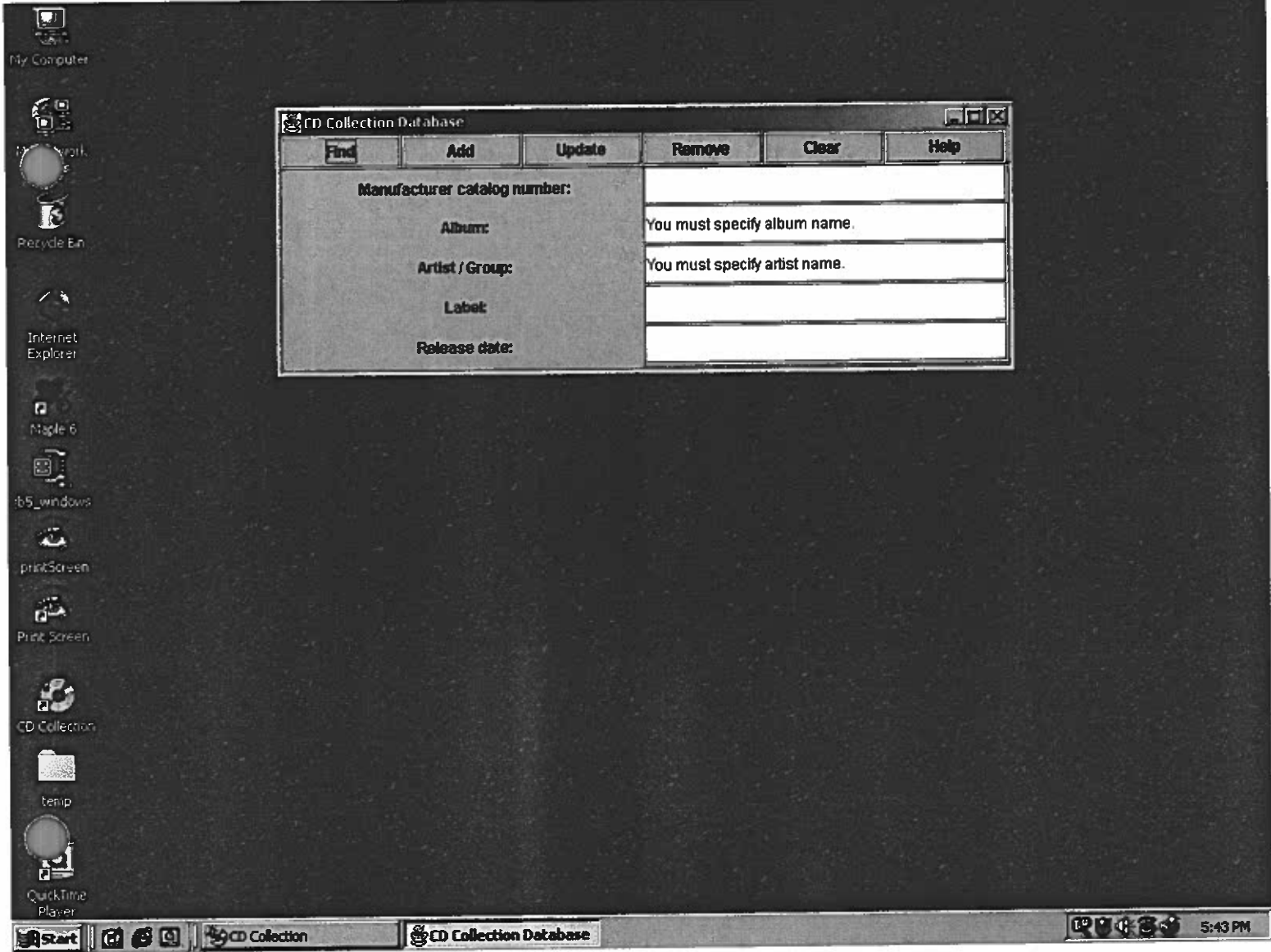
Find	Add	Update	Remove	Clear	Help
Manufacturer catalog number:			46441		
Album:			Revolver		
Artist / Group:			Beatles		
Label:					
Release date:					

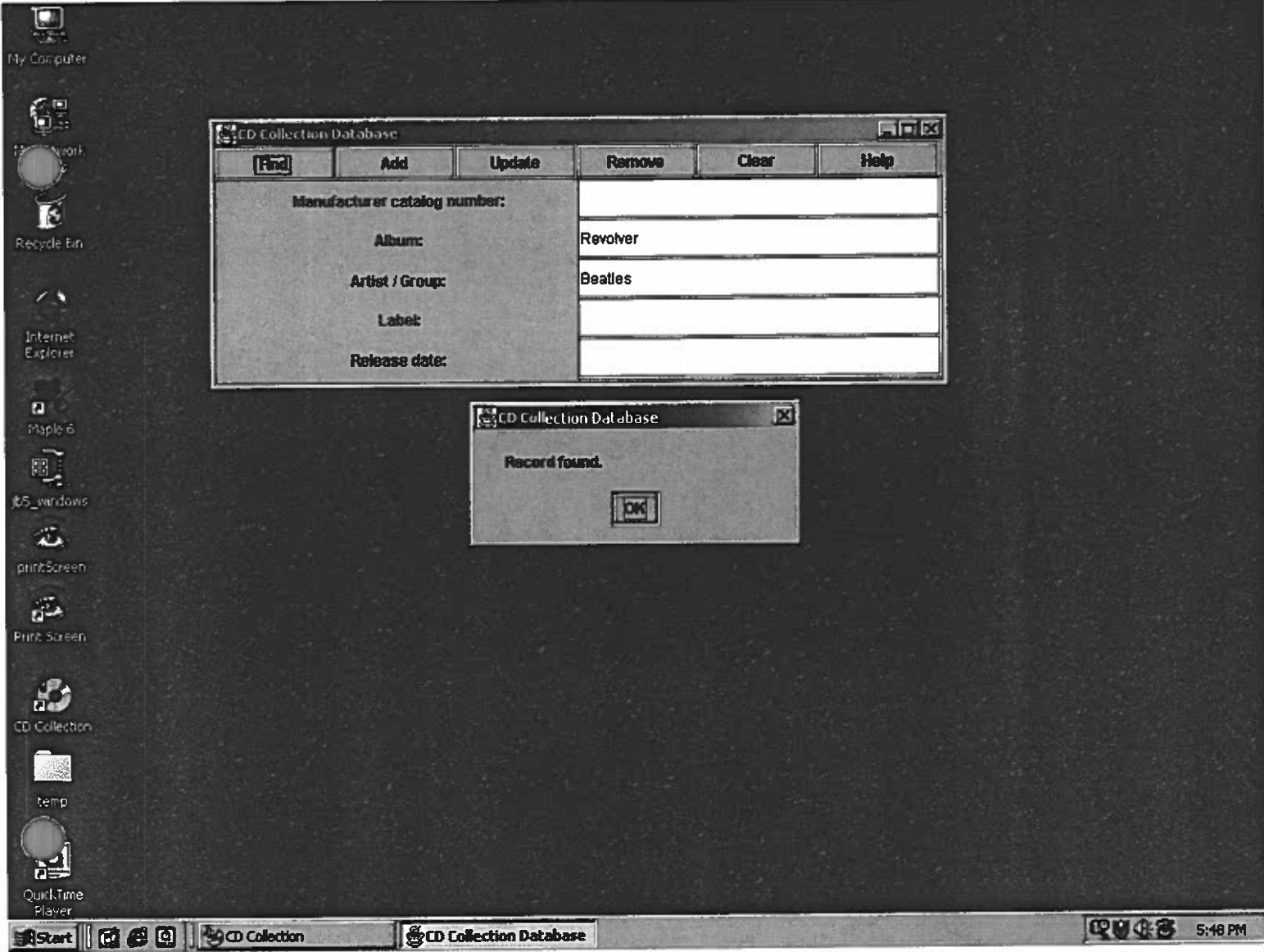
CD Collection Database

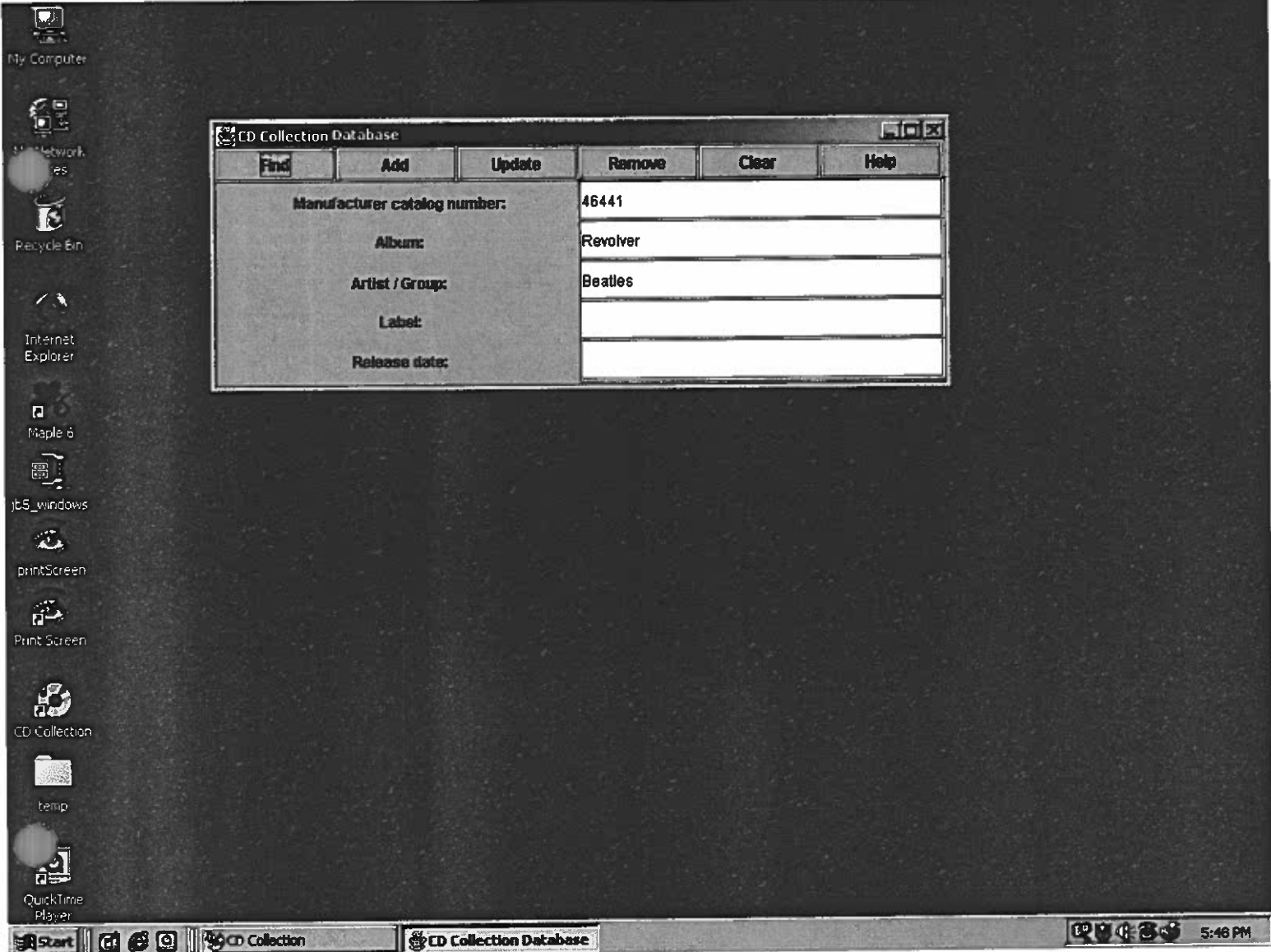
Insertion successful.

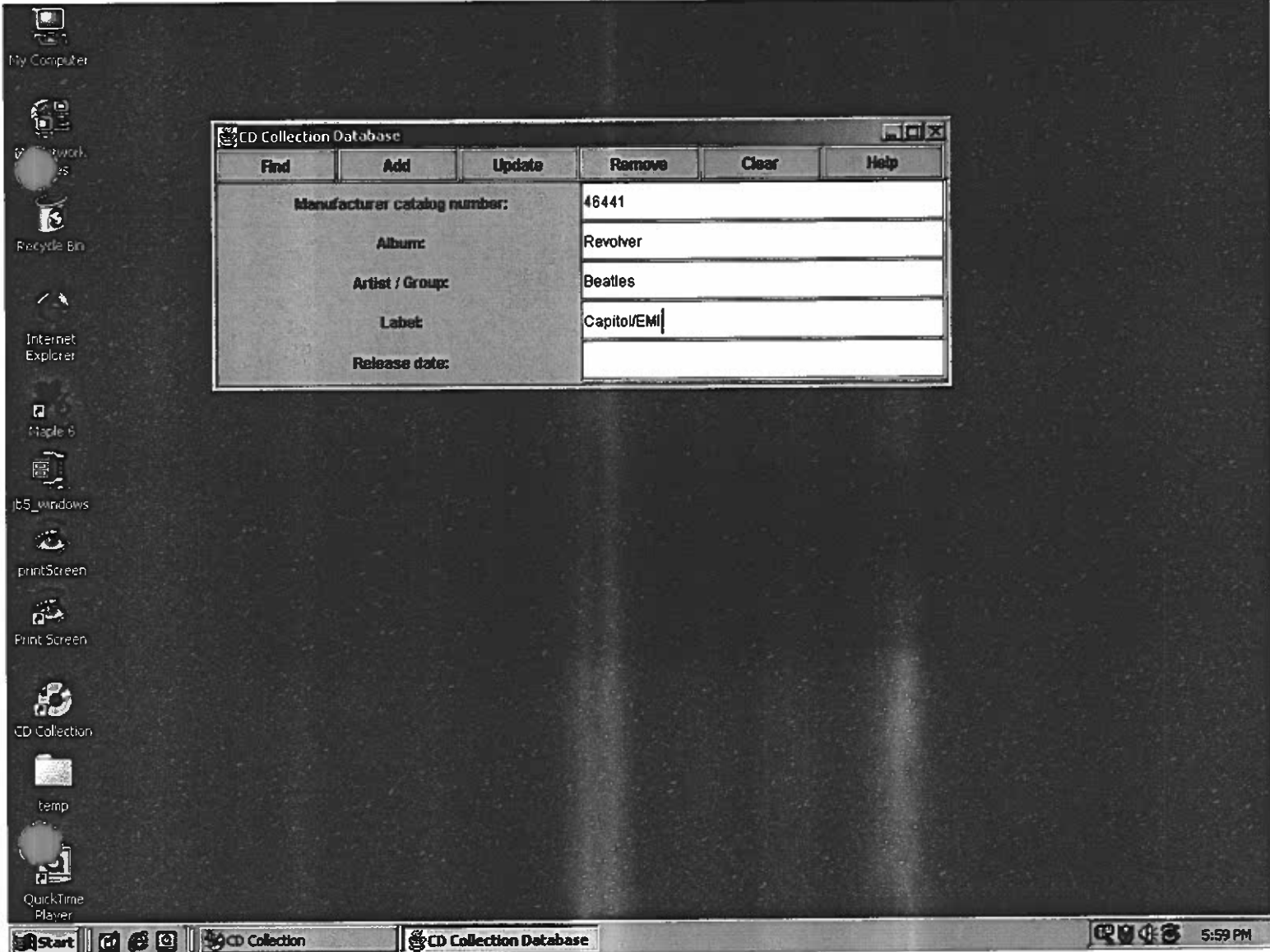
OK

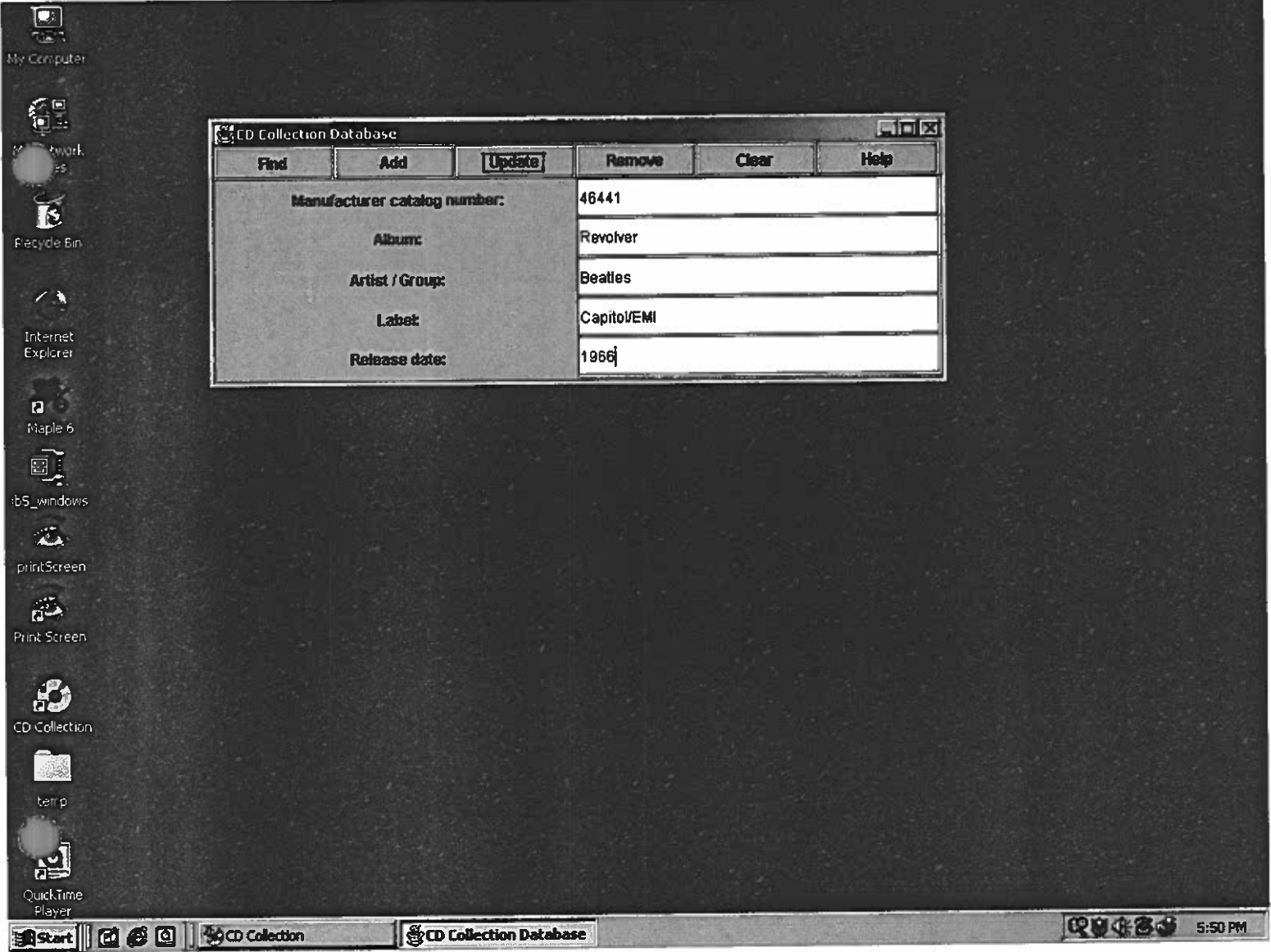




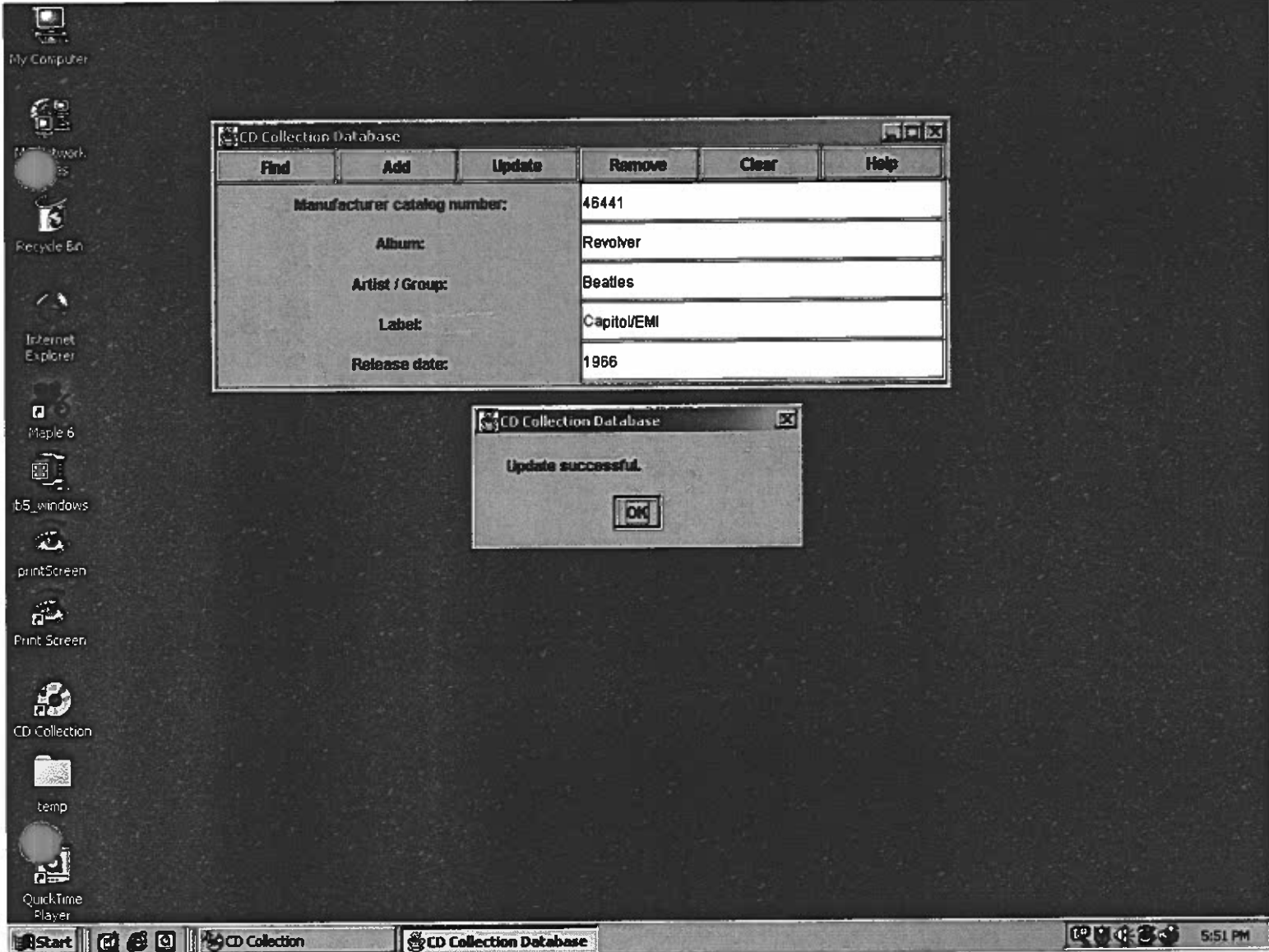


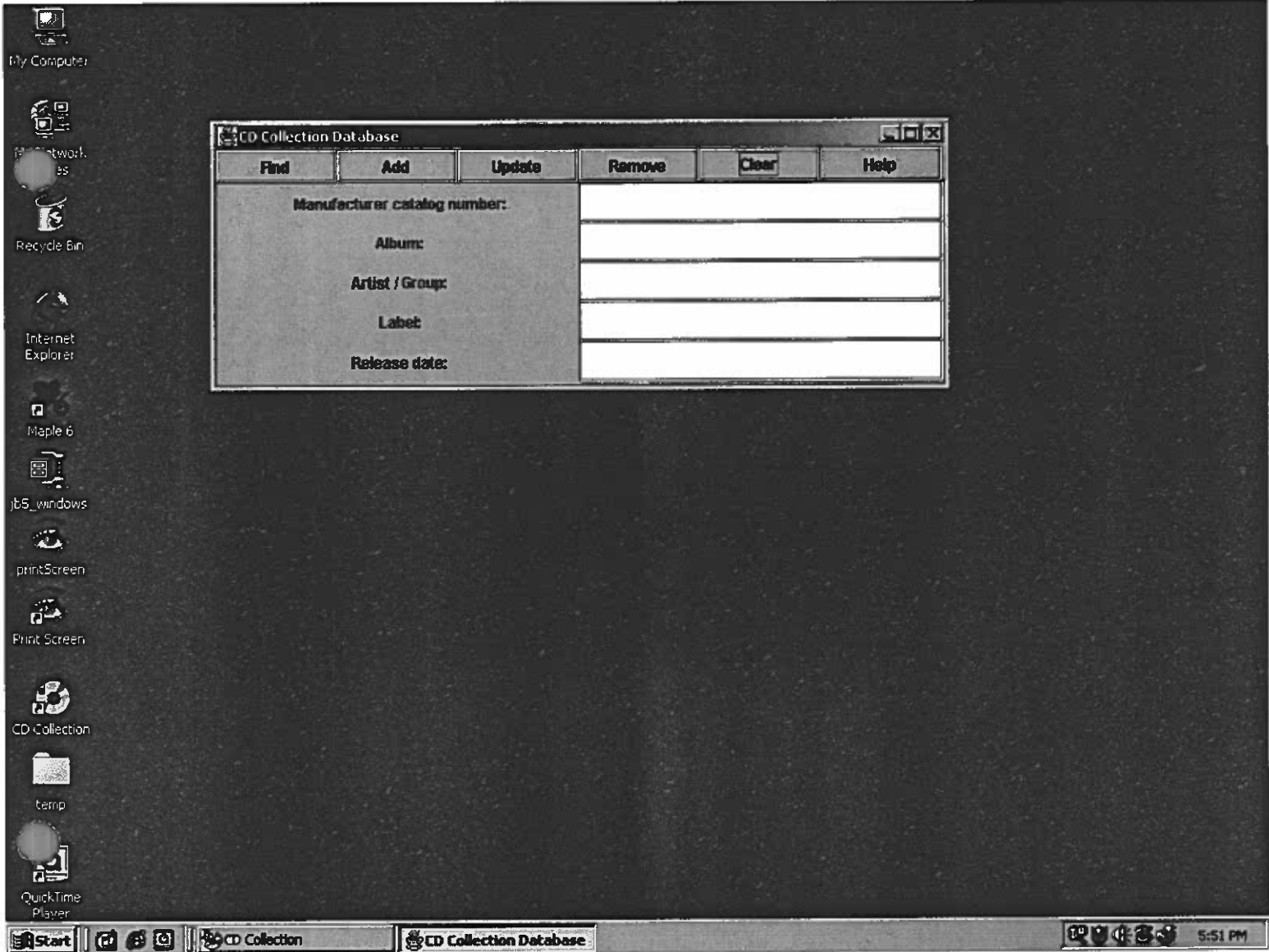


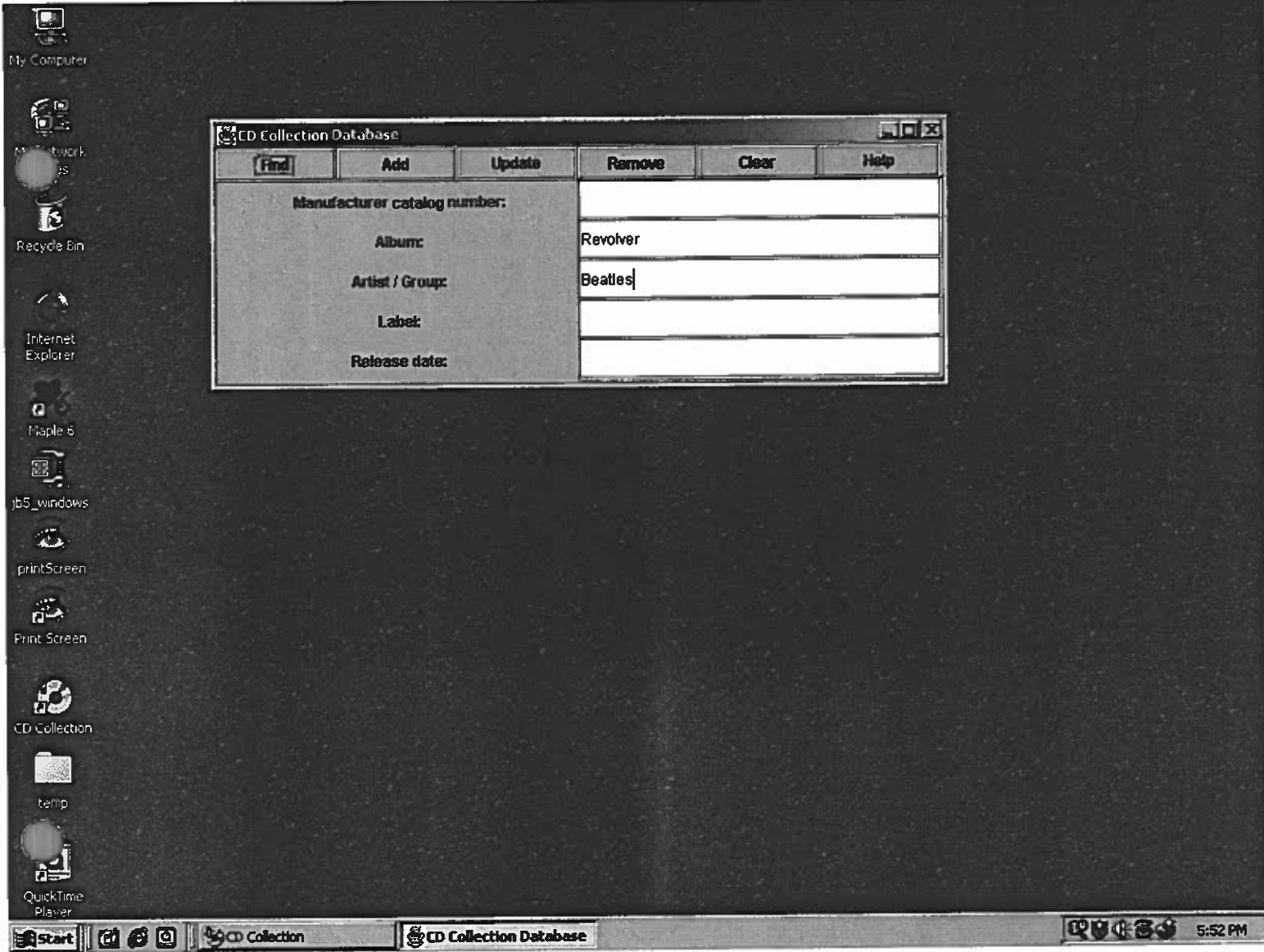


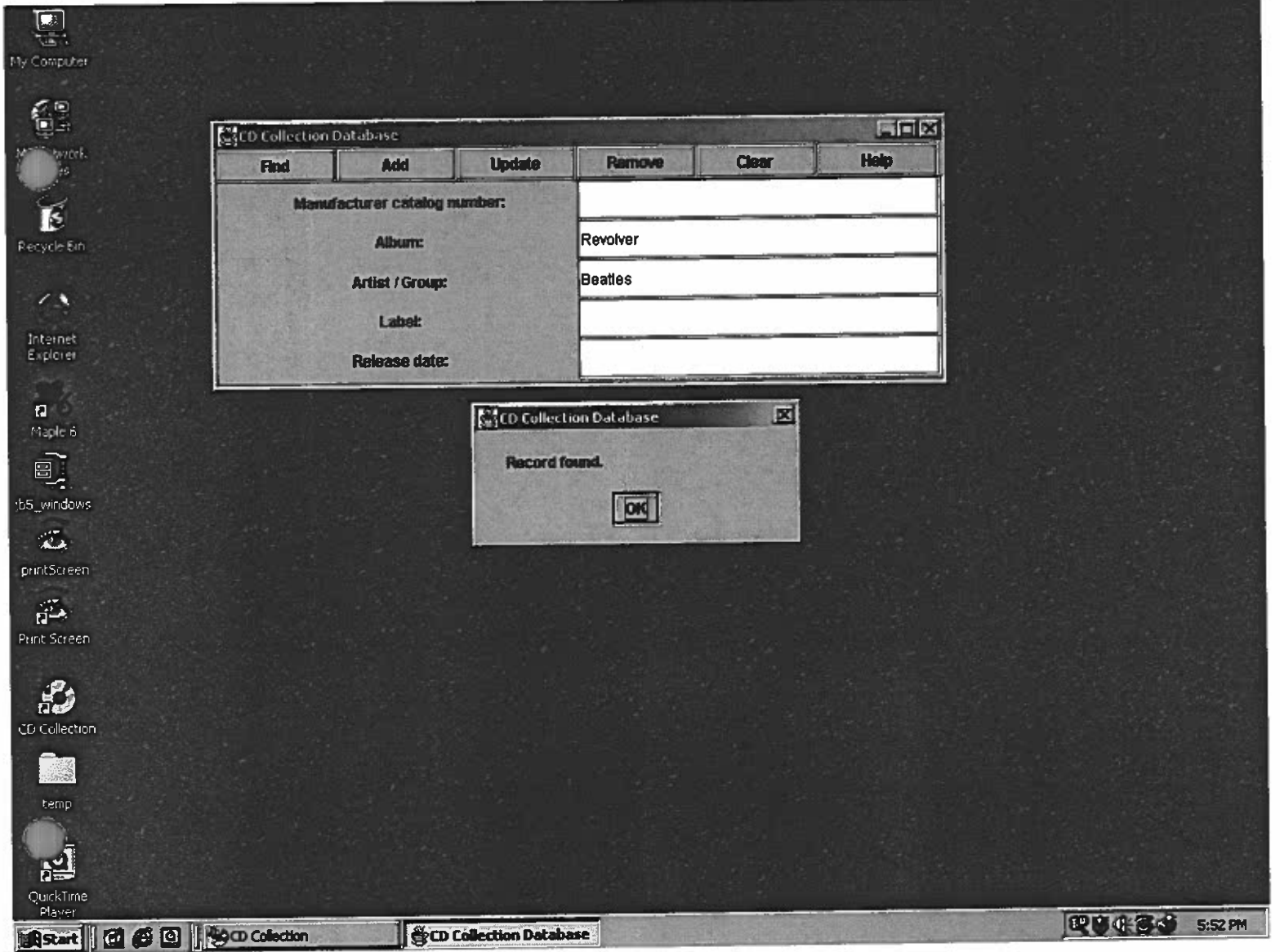


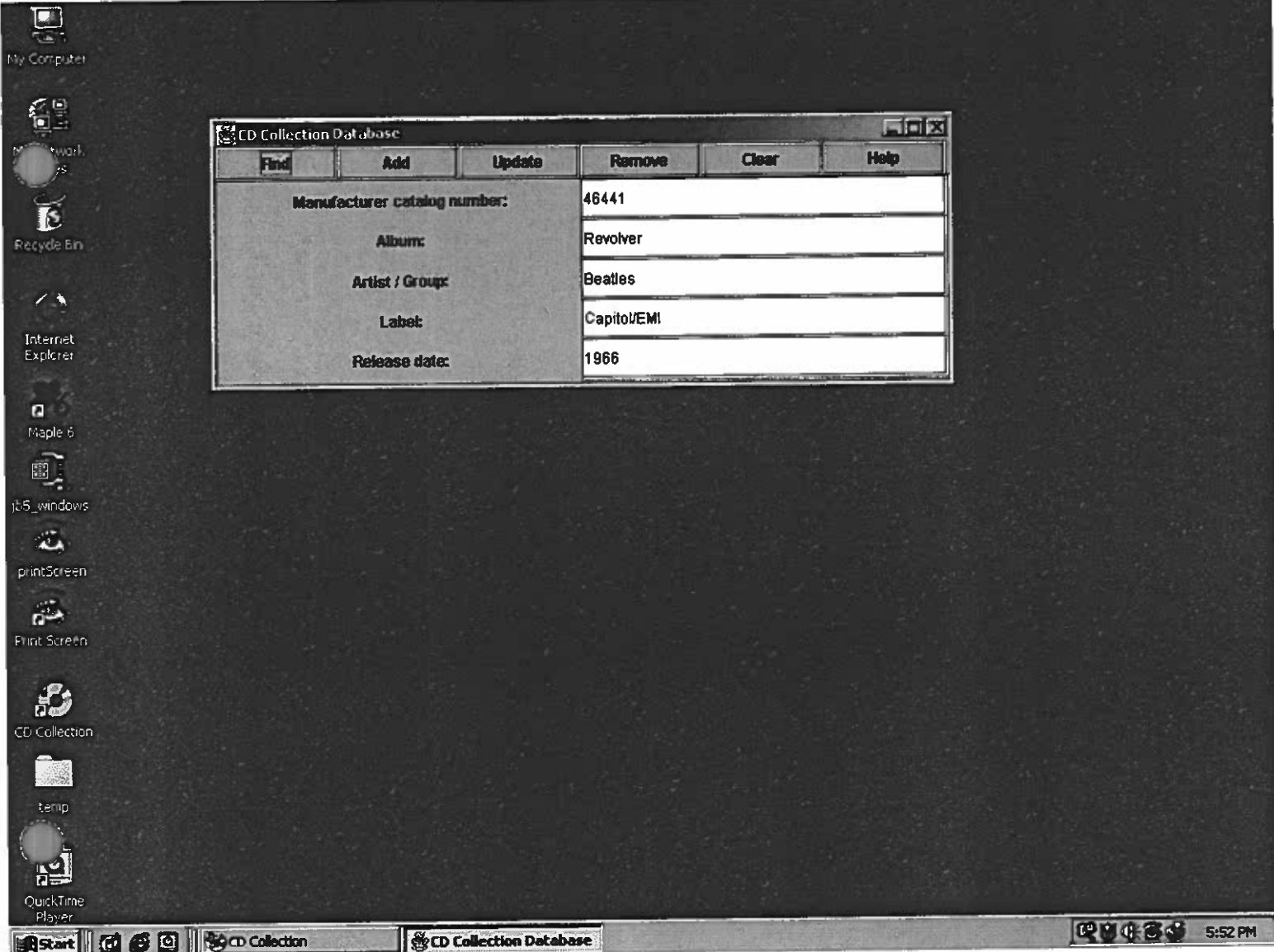
Find	Add	Update	Remove	Clear	Help
Manufacturer catalog number:			46441		
Album:			Revolver		
Artist / Group:			Beatles		
Label:			Capitol/EMI		
Release date:			1966		

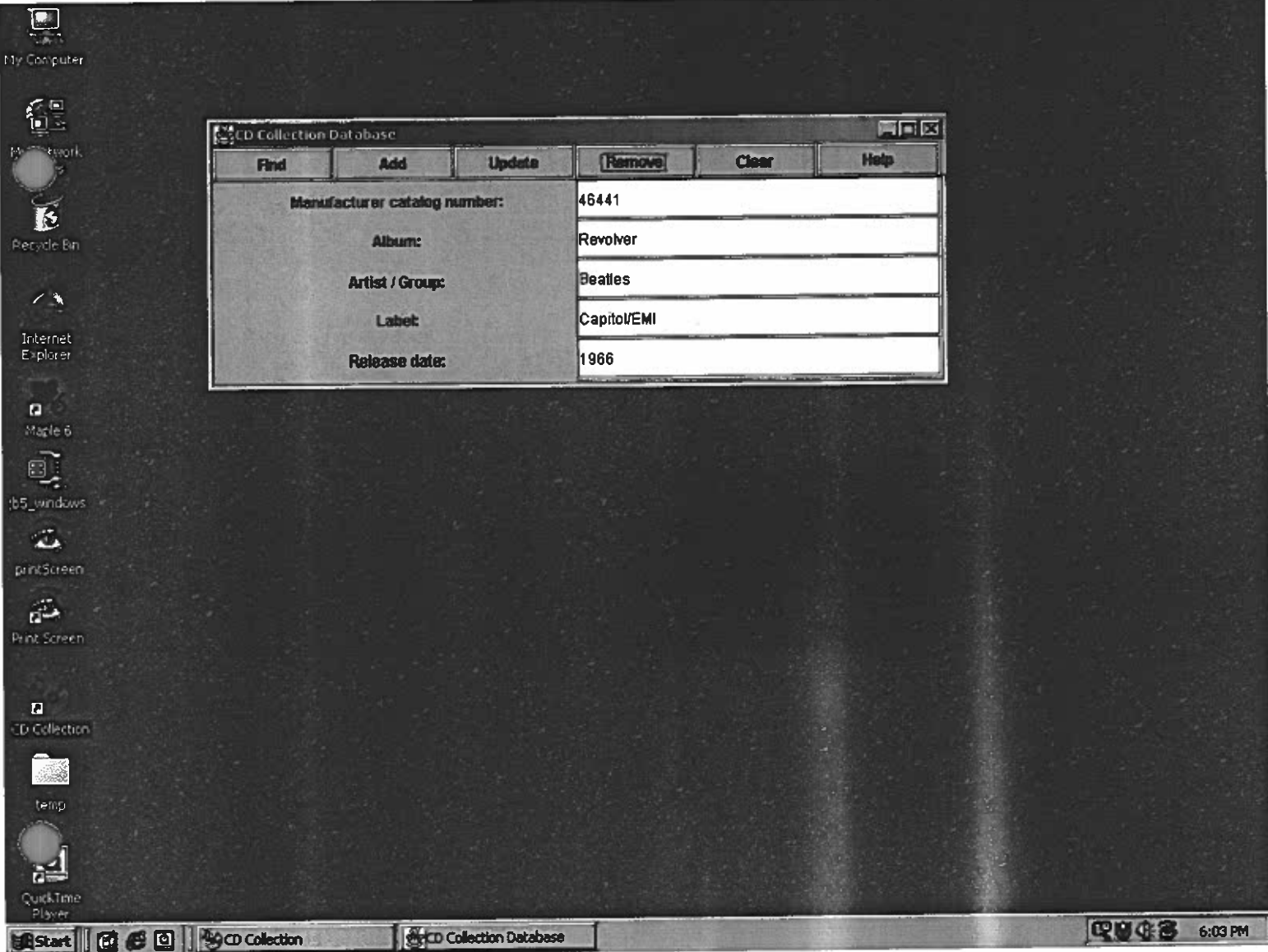












- My Computer
- work
- Recycle Bin
- Internet Explorer
- Maple 6
- tb5_windows
- printScreen
- Print Screen
- CD Collection
- temp
- QuickTime Player

CD Collection Database

Find	Add	Update	Remove	Clear	Help
Manufacturer catalog number:					
Album:					
Artist / Group:					
Label:					
Release date:					

CD Collection Database

Remove successful.

OK

- My Computer
- work
- Recycle Bin
- Internet Explorer
- Maple 6
- 5_windows
- printScreen
- Print Screen
- CD Collection
- temp
- QuickTime Player

CD Collection Database

FindAddUpdateRemoveClearHelp

Manufacturer catalog number:	
Album:	Revolver
Artist / Group:	Beatles
Label:	
Release date:	

- My Computer
- My Recent Places
- Recycle Bin
- Internet Explorer
- Maple 6
- jb5_windows
- printScreen
- Print Screen
- CD Collection
- temp
- QuickTime Player

CD Collection Database

FindAddUpdateRemoveClearHelp

Manufacturer catalog number:	
Album:	Revolver
Artist / Group:	Beatles
Label:	
Release date:	

CD Collection Database

No record found.

OK

THE CD COLLECTION CODE

CDCollection.java

```
// Mirko Mandic
// Class CDCollection definition
// March 4, 2002

import java.sql.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class CDCollection extends JFrame {
    private ControlPanel controls;
    private ScrollingPanel scrollArea;
    private JTextArea output;
    private String url;
    private Connection connect;
    private JScrollPane textpane;

    public CDCollection()
    {
        super( "CD Collection Database" );

        Container c = getContentPane();

        // Start screen layout
        scrollArea = new ScrollingPanel();
        output = new JTextArea( 6, 30 );
        c.setLayout( new BorderLayout() );
        c.add( new JScrollPane( scrollArea ),
              BorderLayout.CENTER );
        textpane = new JScrollPane( output );
        //c.add( textpane, BorderLayout.SOUTH );

        // Set up database connection
        try {
            url = "jdbc:odbc:CDCollection";

            Class.forName( "sun.jdbc.odbc.JdbcOdbcDriver" );
            connect = DriverManager.getConnection( url );
            output.append( "Connection successful\n" );
        }
        catch ( ClassNotFoundException cnfex ) {
            // process ClassNotFoundExceptions here
            cnfex.printStackTrace();
            output.append( "Connection unsuccessful\n" +
                          cnfex.toString() );
        }
        catch ( SQLException sqllex ) {
            // process SQLExceptions here
            sqllex.printStackTrace();
            output.append( "Connection unsuccessful\n" +
                          sqllex.toString() );
        }
    }
}
```

```

                                CDCCollection.java
catch ( Exception ex ) {
    // process remaining Exceptions here
    ex.printStackTrace();
    output.append( ex.toString() );
}

// Complete screen layout
controls =
    new ControlPanel( connect, scrollArea, output);
c.add( controls, BorderLayout.NORTH );

setSize( 600,220 );
show();
}

public static void main( String args[] )
{
    CDCCollection app = new CDCCollection();

    app.addWindowListener(
        new WindowAdapter() {
            public void windowClosing( WindowEvent e )
            {
                System.exit( 0 );
            }
        }
    );
}
}

```

AddRecord.java

```
// Mirko Mandic
// Class AddRecord definition
// March 4, 2002

import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;

public class AddRecord implements ActionListener {
    private ScrollingPanel fields;
    private JTextArea output;
    private Connection connection;

    public AddRecord( Connection c, ScrollingPanel f,
                     JTextArea o )
    {
        connection = c;
        fields = f;
        output = o;
    }

    public void actionPerformed((ActionEvent e) )
    {
        try {
            Statement statement = connection.createStatement();

            if ( ( ! fields.album.getText().equals( "" ) ) &
                ( ! fields.artist.getText().equals( "" ) ) &
                ( ! fields.catalogNumber.getText().equals( "" ) ) &
                ( ! fields.label.getText().equals( "" ) ) &
                ( ! fields.releaseDate.getText().equals( "" ) ) )
            {
                String query = "INSERT INTO CDs" +
                    " VALUES ('" +
                        fields.catalogNumber.getText() + "', '" +
                        fields.album.getText() + "', '" +
                        fields.artist.getText() + "', '" +
                        fields.label.getText() + "', '" +
                        fields.releaseDate.getText() + "');"
                output.append( "\nSending query: " +
                    connection.nativeSQL( query )
                    + "\n" );
                int result = statement.executeUpdate( query );

                if ( result == 1 )
                {
                    output.append( "\nInsertion successful\n" );
                    JOptionPane.showMessageDialog(
                        null, "Insertion successful.", "CD Collection
Database",
                        JOptionPane.PLAIN_MESSAGE );
                }
            }
        }
    }
}
```

AddRecord.java

```
    }
    else {
        output.append( "\nInsertion failed\n" );
        JOptionPane.showMessageDialog(
            null, "Insertion failed.", "CD Collection Database",
            JOptionPane.PLAIN_MESSAGE );

        fields.catalogNumber.setText( "" );
        fields.album.setText( "" );
        fields.artist.setText( "" );
        fields.label.setText( "" );
        fields.releaseDate.setText( "" );
    }
}
else {
    {
        if (fields.album.getText().equals( "" ) )
            fields.album.setText("You must specify album name.");
        if (fields.artist.getText().equals( "" ) )
            fields.artist.setText("You must specify artist name.");
        if (fields.catalogNumber.getText().equals( "" ) )
            fields.catalogNumber.setText("You must specify catalog
number.");
        if (fields.label.getText().equals( "" ) )
            fields.label.setText(" ");
        if (fields.releaseDate.getText().equals( "" ) )
            fields.releaseDate.setText(" ");
    }

    statement.close();
}

catch ( SQLException sqllex ) {
    sqllex.printStackTrace();
    output.append( sqllex.toString() );
}
}
}
```



```

                                FindRecord.java
                                fields.album.setText("You must specify album name." );
                                if ( fields.artist.getText().equals( "" ) )
                                fields.artist.setText("You must specify artist name."
);
                                }
                                }
                                catch ( SQLException sqlex ) {
                                sqlex.printStackTrace();
                                output.append( sqlex.toString() );
                                }
                                }

// Display results of query.
public void display( ResultSet rs )
{
    try {
        while(rs.next())
        {
            int recordNumber = rs.getInt( 1 );
            if ( recordNumber != 0 ) {
                fields.catalogNumber.setText( String.valueOf(
recordNumber));
                fields.album.setText( rs.getString( 2 ) );
                fields.artist.setText( rs.getString( 3 ) );
                fields.label.setText( rs.getString( 4 ) );
                fields.releaseDate.setText( rs.getString( 5 ) );

            }
            else
            {
                output.append( "\nNo record found\n" );
                JOptionPane.showMessageDialog(
                    null, "No record found.", "CD Collection Database",
                    JOptionPane.PLAIN_MESSAGE );
            }
        }
        rs.close();
    }
    catch ( SQLException sqlex ) {
        sqlex.printStackTrace();
        output.append( sqlex.toString() );
    }
}
}

```

UpdateRecord.java

```
// Mirko Mandic
// Class UpdateRecord definition
// March 4, 2002

import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;

public class UpdateRecord implements ActionListener {
    private ScrollingPanel fields;
    private JTextArea output;
    private Connection connection;

    public UpdateRecord( Connection c, ScrollingPanel f,
                        JTextArea o )
    {
        connection = c;
        fields = f;
        output = o;
    }

    public void actionPerformed((ActionEvent e) )
    {
        String query;

        try {
            Statement statement = connection.createStatement();
            if ( ! fields.catalogNumber.getText().equals( "" ) )
            {
                query = "UPDATE CDs SET catalogNumber = '" +
fields.catalogNumber.getText() +
                "', artist = '" + fields.artist.getText() +
                "', album = '" + fields.album.getText() + "', label = '" +
fields.label.getText() + "', releaseDate = '" +
fields.releaseDate.getText() + "' WHERE catalogNumber =
'" +
                fields.catalogNumber.getText() + "'";

                output.append( "\nSending query: " +
connection.nativeSQL( query ) + "\n" );

                int result = statement.executeUpdate( query );

                if ( result == 1 )
                {
                    output.append( "\nUpdate successful\n" );
                    JOptionPane.showMessageDialog(
                        null, "Update successful.", "CD Collection
Database",
                        JOptionPane.PLAIN_MESSAGE );
                }
            }
        }
    }
}
```

UpdateRecord.java

```
else {
    output.append( "\nUpdate failed\n" );
    JOptionPane.showMessageDialog(
        null, "Update failed.", "CD Collection Database",
        JOptionPane.PLAIN_MESSAGE );

    fields.catalogNumber.setText( "" );
    fields.album.setText( "" );
    fields.artist.setText( "" );
    fields.label.setText( "" );
    fields.releaseDate.setText( "" );
}

statement.close();
}
else
{
    output.append( "\nYou may only update an " +
        "existing record. Use Find to " +
        "locate the record, then " +
        "modify the information and " +
        "press Update.\n" );
    JOptionPane.showMessageDialog(
        null, "\nYou may only update an existing record.\n" +
            "\nUse Find to locate the record, then\n" +
            "\nmodify the information and press Update.",
            "CD Collection Database",
            JOptionPane.PLAIN_MESSAGE );
}
}
catch ( SQLException sqllex ) {
    sqllex.printStackTrace();
    output.append( sqllex.toString() );
}
}
}
```

DeleteRecord.java

```
// Mirko Mandic
// Class DeleteRecord definition
// March 4, 2002

import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;

public class DeleteRecord implements ActionListener {
    private ScrollingPanel fields;
    private JTextArea output;
    private Connection connection;

    public DeleteRecord( Connection c, ScrollingPanel f,
                        JTextArea o )
    {
        connection = c;
        fields = f;
        output = o;
    }

    public void actionPerformed((ActionEvent e) )
    {
        String query;

        try {
            Statement statement = connection.createStatement();
            if ( ! fields.catalogNumber.getText().equals( "" ) )
            {
                query = "DELETE FROM CDs WHERE catalogNumber = '" +
                    fields.catalogNumber.getText() + "'";

                output.append( "\nSending query: " +
                    connection.nativeSQL( query ) + "\n" );

                int result = statement.executeUpdate( query );

                if ( result == 1 )
                {
                    fields.catalogNumber.setText( "" );
                    fields.album.setText( "" );
                    fields.artist.setText( "" );
                    fields.label.setText( "" );
                    fields.releaseDate.setText( "" );
                    output.append( "\nDelete successful\n" );
                    JOptionPane.showMessageDialog(
                        null, "Remove successful.", "CD collection
Database",
                        JOptionPane.PLAIN_MESSAGE );
                }
            }
            else {
```

```

DeleteRecord.java
output.append( "\nDelete failed\n" );
JOptionPane.showMessageDialog(
    null, "Delete failed.", "CD Collection Database",
    JOptionPane.PLAIN_MESSAGE );

fields.catalogNumber.setText( "" );
fields.album.setText( "" );
fields.artist.setText( "" );
fields.label.setText( "" );
fields.releaseDate.setText( "" );
}

statement.close();
}
else
{
    output.append( "\nYou may only delete an " +
        "existing record. Use Find to " +
        "locate the record, then " +
        "press Delete.\n" );
    JOptionPane.showMessageDialog(
        null, "You may only remove an existing record.\n" +
            "\nUse Find to locate the record, then press
Remove.\n",
        "CD Collection Database",
        JOptionPane.PLAIN_MESSAGE );
}
}
catch ( SQLException sqlx ) {
    sqlx.printStackTrace();
    output.append( sqlx.toString() );
}
}
}

```

clearFields.java

```
// Mirko Mandic
// Class ClearFields definition
// March 4, 2002

import java.awt.*;
import java.awt.event.*;

public class ClearFields implements ActionListener {
    private ScrollingPanel fields;

    public ClearFields( ScrollingPanel f )
    {
        fields = f;
    }

    public void actionPerformed((ActionEvent e) )
    {
        fields.catalogNumber.setText( "" );
        fields.album.setText( "" );
        fields.artist.setText( "" );
        fields.label.setText( "" );
        fields.releaseDate.setText( "" );
    }
}
```

Help.java

```
// Mirko Mandic  
// Class Help definition  
// March 4, 2002
```

```
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;
```

```
public class Help implements ActionListener {  
    private JTextArea output;  
  
    public Help( JTextArea o )  
    {  
        output = o;  
    }  
  
    public void actionPerformed((ActionEvent e) )  
    {  
        output.append( "\nClick Find to locate a record.\n" +  
            "Click Add to insert a new record.\n" +  
            "Click Update to update " +  
            "the information in a record.\n" +  
            "Click Clear to empty" +  
            " the textfields.\n" );  
        JOptionPane.showMessageDialog(  
            null, "Click Find to locate a record.\n" +  
                "Click Add to insert a new record.\n" +  
                "Click Update to update " +  
                "the information in a record.\n" +  
                "Click Delete to delete a record.\n" +  
                "Click Clear to empty" +  
                " the textfields.\n",  
            "CD Collection Database", JOptionPane.PLAIN_MESSAGE );  
    }  
}
```

ControlPanel.java

```
// Mirko Mandic
// Class ControlPanel definition
// March 4, 2002

import java.awt.*;
import java.awt.event.*;
import java.sql.*;
import javax.swing.*;

public class ControlPanel extends JPanel {
    private JButton findAlbum, addAlbum,
        updateAlbum, deleteAlbum, clear, help;

    public ControlPanel( Connection c, ScrollingPanel s,
        JTextArea t )
    {
        setLayout( new GridLayout( 1, 6 ) );

        findAlbum = new JButton( "Find" );
        findAlbum.addActionListener( new FindRecord( c, s, t ) );
        add( findAlbum );

        addAlbum = new JButton( "Add" );
        addAlbum.addActionListener( new AddRecord( c, s, t ) );
        add( addAlbum );

        updateAlbum = new JButton( "Update" );
        updateAlbum.addActionListener(
            new UpdateRecord( c, s, t ) );
        add( updateAlbum );

        deleteAlbum = new JButton( "Remove" );
        deleteAlbum.addActionListener( new DeleteRecord( c, s, t ) );
        add( deleteAlbum );

        clear = new JButton( "Clear" );
        clear.addActionListener( new ClearFields( s ) );
        add( clear );

        help = new JButton( "Help" );
        help.addActionListener( new Help( t ) );
        add( help );
    }
}
```

ScrollingPanel.java

```
// Mirko Mandic
// Class ScrollingPanel
// March 4, 2002

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ScrollingPanel extends JPanel {
    private JPanel labelPanel, fieldsPanel;
    private String labels[] =
        { "Catalog number:", "Album:", "Artist / Group:",
          "Label:", "Release date:" };
    JTextField catalogNumber, album, artist, label, releaseDate;

    public ScrollingPanel()
    {
        // Label panel
        labelPanel = new JPanel();
        labelPanel.setLayout(
            new GridLayout( labels.length, 1 ) );

        ImageIcon ii = new ImageIcon( "images/icon.jpg" );

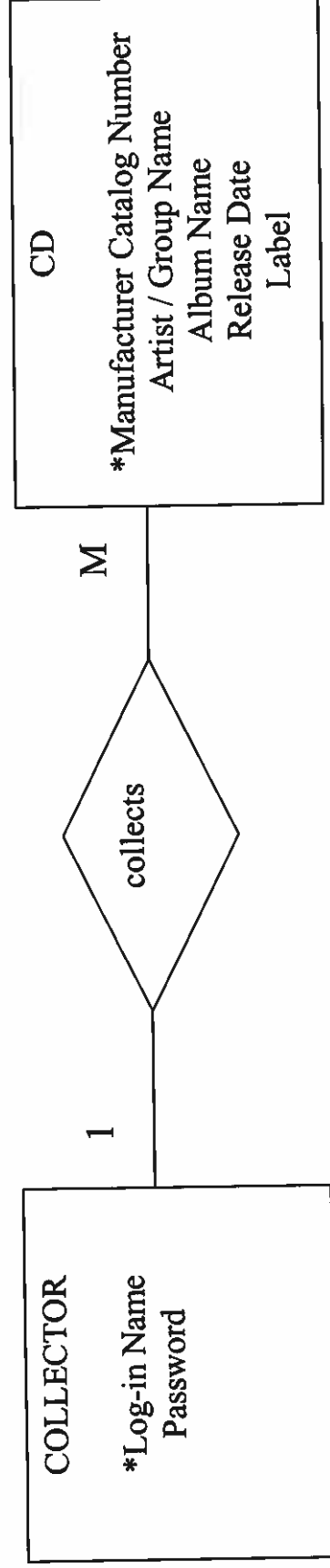
        for ( int i = 0; i < labels.length; i++ )
            labelPanel.add( new JLabel( labels[ i ], ii, 0 ) );

        // TextField panel
        fieldsPanel = new JPanel();
        fieldsPanel.setLayout(
            new GridLayout( labels.length, 1 ) );
        catalogNumber = new JTextField( 20 );
        // number.setEditable( false );
        fieldsPanel.add( catalogNumber );
        album = new JTextField( 20 );
        fieldsPanel.add( album );
        artist = new JTextField( 20 );
        fieldsPanel.add( artist );
        label = new JTextField( 20 );
        fieldsPanel.add( label );
        releaseDate = new JTextField( 20 );
        fieldsPanel.add( releaseDate );

        setLayout( new GridLayout( 1, 2 ) );
        add( labelPanel );
        add( fieldsPanel );
    }
}
```

APPENDICES

APPENDIX A – CD COLLECTION ER DIAGRAM (EXPANDED)



APPENDIX B - CANDIDATE PAYBACK ANALYSIS

Candidate 1:

Cost to develop = \$ 5,000.00

Payback period = 1 year

Discount rate = 10 %

Present value = $1 / (1 + .1)^1 = .909091$

Net present value = $5,000.00 * .909091 = \$ 4,545.00$

Candidate 2:

Cost to develop = \$ 13,000.00

Payback period = 3 years

Discount rate = 10 %

Present value = $1 / (1 + .1)^3 = .751315$

Net present value = $13,000.00 * .751315 = \$ 9,767.00$

**APPENDIX C – ENTITIES TO THIRD NORMAL FORM
(EXPANDED)**

Collector (*Sign-in Name, Password)

CD (*Manufacturer Catalog Number, Artist/Group Name, Album Name,
Release Date, Label, Sign-in Name)